Department of Computer Science

Master Degree in Data Science and Business Informatics

Optimization for Data Science

Group 12 - Project 12

Professor:

Antonio Frangioni

Group members:

Capodagli Chiara (693737)

Roselli Riccardo (614720)

ACADEMIC YEAR 2025/2026

Contents

1 Introduction **1**
1.1 Problem Formulation and Gradients . . . 2
1.2 Algorithms . . . 3
1.2.1 A1 — Momentum Descent (Heavy Ball) . . . 3
1.2.2 A2 — Smoothed Gradient Method . . . 5
1.3 Objective . . . 8

2 Implementation and Numerical Validation **8**
2.1 Data Generation . . . 9
2.2 ELM Feature Extraction . . . 9
2.3 Efficient Computation of Spectral Constants . . . 9
2.4 Reference Solver and Gap Computation . . . 9
2.5 Numerical Verification of Gradients . . . 10
2.6 Huber Smoothing and Approximation Error Control . . . 10
2.7 Optimization Tracking and Stopping Criteria . . . 10
2.8 Implementation of A1 — Heavy-Ball Momentum Method . . . 11
2.9 Implementation of A2 — Nesterov Smoothing with FISTA . . . 13

3 Experiments **14**
3.1 Experiment 1: Convergence Behaviour . . . 14
3.2 Experiment 2: Hyperparameter Sensitivity . . . 16
3.3 Experiment 3: Scalability . . . 18
3.4 Experiment 4: Algorithm Comparison . . . 20
3.5 Experiment 5: Sparsity Analysis . . . 21
3.6 Experiment 6: Full Multi-Column Problem . . . 22
3.7 Experiment 7: Quantitative Rate Analysis . . . 23
3.8 Summary and Discussion . . . 26

1 Introduction

This report studies the training of an *Extreme Learning Machine* (ELM), a neural network with a single hidden layer of the form

$$z = W_2\sigma(W_1x),$$

where $\sigma(\cdot)$ is an elementwise activation function. In the ELM framework the hidden-layer weights W_1 are randomly generated and kept fixed throughout training, while only the output weight matrix W_2 is optimized. Given a dataset of N samples with input matrix $X \in \mathbb{R}^{N\times d}$ and target matrix $Y \in \mathbb{R}^{N\times p}$, the hidden-layer feature matrix is computed once as

$$H = \sigma(XW_1^T) \in \mathbb{R}^{N\times m},$$

where m denotes the number of hidden units. Since W_1 is fixed, the matrix H remains constant during the optimization process. The training problem addressed in this project is the matrix Lasso objective

$$h(W_2) = \|HW_2^T - Y\|_F^2 + \lambda\|W_2\|_1,$$

where $W_2 \in \mathbb{R}^{p\times m}$ is the optimization variable, $\lambda > 0$ is the regularization parameter, and $\|\cdot\|_F$ and $\|\cdot\|_1$ denote the Frobenius and elementwise L_1 norms respectively. Both the squared Frobenius loss and the L_1 penalty are separable across the rows of W_2, so the problem decomposes into p independent single-column Lasso sub-problems: for each output $j \in \{1,\dots,p\}$ we solve

$$\min_{w\in\mathbb{R}^m} \; h(w) := \|Hw - y_j\|_2^2 + \lambda\|w\|_1,$$

where $y_j \in \mathbb{R}^N$ is the j-th column of Y and w is the j-th row of W_2 (viewed as a column vector). All derivations and algorithms in this report are presented on this per-column formulation; the full matrix problem is recovered by solving the p columns independently.

We write the per-column objective in composite form

$$h(w) = g(w) + r(w), \qquad g(w) = \|Hw - y\|_2^2, \qquad r(w) = \lambda\|w\|_1,$$

where g is a convex quadratic (smooth, with Lipschitz-continuous gradient) and r is convex but nonsmooth. This composite structure naturally motivates the use of first-order optimization methods. In this project we consider two approaches: the first algorithm (A1) is a momentum-based gradient method using the heavy-ball scheme originally proposed by *Polyak 1964*, which introduces an inertial term to accelerate convergence in smooth optimization problems and has been widely used in modern deep learning (e.g., *Sutskever et al. 2013*). The second algorithm (A2) belongs to the class of smoothed gradient methods introduced by *Nesterov 2005*, which approximate the nonsmooth L_1 term with a smooth surrogate, allowing efficient gradient-based optimization. Finally, the project requirements involve medium-scale instances (at least 10^4 weights and 10^3 inputs), making iterative first-order methods a natural choice. All algorithms are implemented from scratch, as no off-the-shelf solvers are allowed. Figure 1 illustrates the architecture of the ELM.

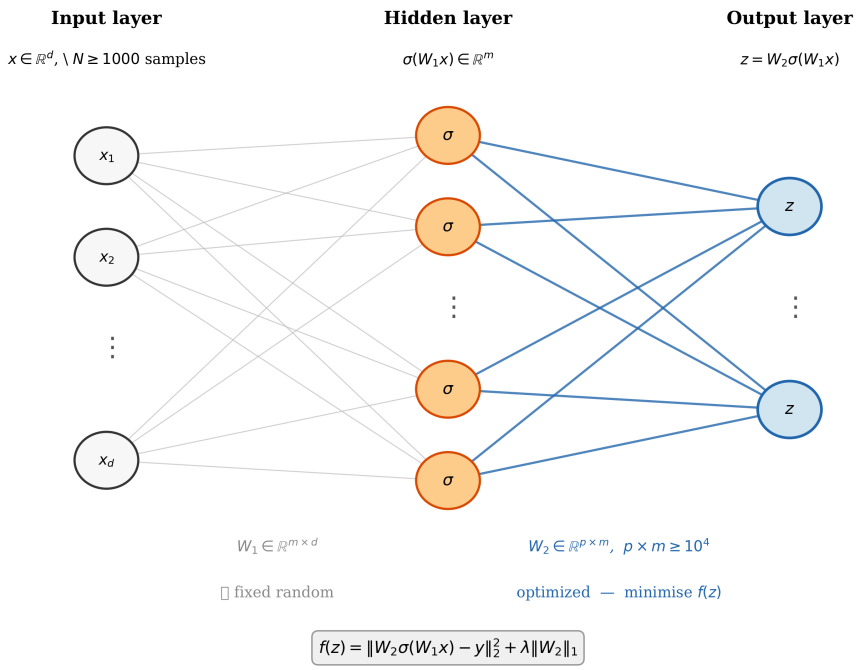

Figure 1: Architecture of the Extreme Learning Machine. The hidden-layer weights W_1 are fixed and random, only the output weights W_2 are optimized.

1.1 Problem Formulation and Gradients

Following the per-column decomposition introduced above, all derivations in this section are presented on a single sub-problem

$$\min_{w \in \mathbb{R}^m} \; h(w) = g(w) + r(w), \qquad g(w) = \|Hw - y\|_2^2, \qquad r(w) = \lambda \|w\|_1,$$

where $y \in \mathbb{R}^N$ denotes one fixed column of Y and $w \in \mathbb{R}^m$ is the corresponding column of W_2^T. Since W_1 (and hence H) is fixed, no backpropagation through the hidden layer is required.

Gradient of the smooth term

Expanding

$$g(w) = (Hw - y)^T (Hw - y) = w^T H^T H w - 2\, y^T H w + y^T y$$

and differentiating with respect to w yields

$$\nabla g(w) = 2\, H^T (Hw - y) \in \mathbb{R}^m.$$

Lipschitz constant of the gradient

The gradient of g is Lipschitz continuous with constant

$$L_g = 2\|H\|_2^2 = 2\lambda_{\max}(H^T H),$$

where $\|H\|_2$ denotes the spectral norm of H (largest singular value) and $\lambda_{\max}(H^T H)$ is the largest eigenvalue of $H^T H \in \mathbb{R}^{m \times m}$. This quantity can be computed explicitly or upper-bounded using the Frobenius norm,

$$L_g \leq 2\|H\|_F^2,$$

which avoids computing the spectral norm directly.

Strong convexity of the smooth term

The function g is strongly convex with parameter

$$\mu_g = 2\lambda_{\min}(H^T H),$$

where $\lambda_{\min}(H^T H)$ is the smallest eigenvalue of $H^T H$. If H has full column rank (i.e. $N \geq m$ and the columns of H are linearly independent), then $\lambda_{\min}(H^T H) > 0$ and g is strongly convex. If $m > N$ or the columns of H are linearly dependent, then $\lambda_{\min}(H^T H) = 0$ and g is convex but not strongly convex.

The *condition number* of the smooth part is defined as

$$\kappa = \frac{L_g}{\mu_g} = \frac{\lambda_{\max}(H^T H)}{\lambda_{\min}(H^T H)}.$$

This quantity measures the elongation of the level sets of g: a large κ indicates that the curvature varies significantly across different directions, making optimization harder. As will be discussed in the next section, the convergence speed of momentum methods depends critically on κ.

Subgradient of the nonsmooth term

For the nonsmooth regularization term $r(w) = \lambda\|w\|_1$, a subgradient is defined elementwise as

$$(\partial r(w))_i = \lambda\, s_i, \qquad s_i = \begin{cases} \operatorname{sign}(w_i) & \text{if } w_i \neq 0, \\ [-1, 1] & \text{if } w_i = 0. \end{cases}$$

At $w_i = 0$ we select the minimum-norm subgradient $s_i = 0$, which is the convention used throughout the report (and coincides with `np.sign(0) = 0` in the NumPy implementation).

Throughout this report the symbol σ is reserved exclusively for the activation function, while the largest eigenvalue of a matrix is denoted by $\lambda_{\max}$.

1.2 Algorithms

This section describes the two first-order algorithms used to minimize the composite objective

$$h(w) = g(w) + r(w), \qquad g(w) = \|Hw - y\|_2^2, \qquad r(w) = \lambda\|w\|_1,$$

where g is smooth convex and r is convex but nonsmooth. Algorithm A1 is a momentum-based method, provided in both a subgradient variant and a proximal variant; algorithm A2 replaces the nonsmooth term r with a smooth Huber surrogate so that an accelerated gradient scheme can be applied.

1.2.1 A1 — Momentum Descent (Heavy Ball)

The heavy-ball method proposed by Polyak 1964 augments the standard gradient descent step with an inertial term proportional to the previous displacement. This idea has later been widely adopted in neural network optimization, where momentum is known to significantly accelerate training Sutskever et al. 2013.

Subgradient variant. Let w_t denote the iterate at step t and let $v_t \in \mathbb{R}^m$ be the velocity. With $\nabla g(w_t)$ the gradient of the smooth part and $s_t \in \partial r(w_t)$ a chosen subgradient, the generalized (sub)gradient of the composite objective is

$$\nabla h(w_t) = \nabla g(w_t) + s_t, \qquad s_t = \lambda \operatorname{sign}(w_t)$$

(using the minimum-norm convention $s_i = 0$ at $w_i = 0$). The classical-momentum (CM) update is

$$v_{t+1} = \beta\, v_t - \eta\, \nabla h(w_t), \qquad w_{t+1} = w_t + v_{t+1},$$

where $\eta > 0$ is the step size, $\beta \in [0, 1)$ is the momentum coefficient, and $v_0 = 0$. Setting $\beta = 0$ recovers plain subgradient descent. The velocity term accumulates an exponentially decaying average of past (sub)gradient directions, accelerating progress along persistent descent directions while damping oscillations where the gradient changes sign frequently.

In the Nesterov-style lookahead (NAG) variant, the (sub)gradient is evaluated at the extrapolated point $w_t + \beta v_t$:

$$v_{t+1} = \beta\, v_t - \eta\, \nabla h(w_t + \beta v_t), \qquad w_{t+1} = w_t + v_{t+1}.$$

The lookahead anticipates the effect of the momentum step and improves stability along directions of high curvature.

Convergence theory for smooth strongly convex objectives. The convergence analysis of the heavy-ball method is best understood in the smooth strongly convex case. Consider minimizing a function φ with L-Lipschitz gradient and strong convexity parameter $\mu > 0$, so that the condition number is $\kappa = L/\mu$. Plain gradient descent with step size $\eta = 1/L$ achieves the linear rate

$$\varphi(w_t) - \varphi(w^*) \le C \left(\frac{\kappa - 1}{\kappa + 1} \right)^{2t},$$

requiring $O(\kappa \log(1/\varepsilon))$ iterations to reach ε-accuracy. Polyak (1964) showed that, on a smooth strongly convex quadratic with condition number κ, the optimal heavy-ball parameters are

$$\eta^* = \frac{4}{(\sqrt{L} + \sqrt{\mu})^2}, \qquad \beta^* = \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^2,$$

and yield the improved contraction factor $\rho^* = (\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$, requiring only $O(\sqrt{\kappa} \log(1/\varepsilon))$ iterations to reach ε-accuracy.

Applicability to our problem. These guarantees do not apply directly in our composite setting. Writing the condition number of g as

$$\kappa = \frac{L_g}{\mu_g} \qquad \text{(equivalently } \lambda_{\max}(H^T H)/\lambda_{\min}(H^T H)\text{, since the factors of 2 cancel)},$$

two obstructions arise: (i) $g(w) = \|Hw - y\|_2^2$ is convex quadratic but may fail to be strongly convex when $\mu_g = 2\lambda_{\min}(H^T H) = 0$, in which case $\kappa = \infty$ and the Polyak parameters (η^*, β^*) are not defined; and (ii) the regularizer $r(w) = \lambda \|w\|_1$ is nonsmooth, so the composite objective has no Lipschitz-continuous gradient. In this nonsmooth subgradient setting, the heavy-ball iteration does not inherit the accelerated rate of the smooth case: standard subgradient methods converge at the worst-case rate $O(1/\sqrt{k})$, and adding momentum does not improve this bound in general. Momentum is nevertheless widely used in practice due to its strong empirical performance.

NAG vs. CM: Sutskever Theorem 2.1. On a quadratic objective $q(w) = \frac{1}{2} w^T A w$, Theorem 2.1 of Sutskever et al. 2013 shows that NAG operates along each eigendirection of A with an effective momentum coefficient $\beta(1 - \lambda_i\, \eta)$, where λ_i is the eigenvalue of A and η is the step size. High-curvature directions thus receive lower effective momentum, which prevents oscillations and allows NAG to tolerate larger values of β than CM. On our problem this advantage is modest at $\beta = 0.9$ (CM and NAG perform similarly) but becomes more pronounced at $\beta \ge 0.95$, where CM tends to oscillate.

Proximal variant. Instead of using a subgradient of r, we can split the update: the gradient step drives the velocity using *only* the smooth gradient ∇g, while the nonsmooth term is handled by the proximal operator applied after the momentum step. For the CM variant,

$$v_{t+1} = \beta\, v_t - \eta\, \nabla g(w_t), \qquad w_{t+1} = \mathrm{prox}_{\eta\lambda}(w_t + v_{t+1}),$$

and for the NAG variant,

$$v_{t+1} = \beta\, v_t - \eta\, \nabla g(w_t + \beta v_t), \qquad w_{t+1} = \mathrm{prox}_{\eta\lambda}(w_t + v_{t+1}),$$

where the proximal operator of $\eta r = \eta\lambda\|\cdot\|_1$ is the elementwise soft-thresholding map

$$\mathrm{prox}_{\eta\lambda}(u)_i = \mathrm{sign}(u_i)\ \max\big(|u_i| - \eta\lambda,\, 0\big).$$

The threshold is $\eta\lambda$ (step size times regularization parameter), *not* λ alone: any component with $|u_i| < \eta\lambda$ is mapped to exact zero, so the proximal variant can produce bit-exact sparsity in a single step, in contrast to the subgradient approach which only approaches zero asymptotically.

Proximal-momentum conflict. The velocity v accumulates an exponentially decaying history of past gradient steps, whereas the proximal operator is memoryless: it inspects only the current point $w_t + v_{t+1}$ and thresholds it. This mismatch means that even when the smooth gradient at a zero component has been small for many iterations, the accumulated velocity can still carry that component past the soft-thresholding boundary in the next step, so the proximal operator does not zero it out. The result is a *proximal-momentum plateau*: exact sparsity is not attained while β remains large, and the optimality gap stalls at a level that depends on β and on the problem's conditioning (analyzed empirically in §3).

Proximal gradient convergence theory. In the pure proximal gradient setting ($\beta = 0$, no momentum), classical composite-minimization theory applies. The hypotheses required are:

- g convex with L_g-Lipschitz gradient ✓(here g is a convex quadratic with $L_g = 2\lambda_{\max}(H^T H)$);
- r convex, proper, and lower semi-continuous ✓(here $r = \lambda\|\cdot\|_1$, the canonical example).

With step $\eta = 1/L_g$, proximal gradient descent satisfies

$$h(w_k) - h^* \le \frac{L_g\,\|w_0 - w^*\|^2}{2k} = O\big(\tfrac{1}{k}\big)\,.$$

When g is also strongly convex with parameter $\mu_g > 0$, the rate becomes linear,

$$h(w_k) - h^* \le \left(1 - \frac{\mu_g}{L_g}\right)^k \big(h(w_0) - h^*\big),$$

corresponding to $O(\kappa\log(1/\varepsilon))$ iterations with $\kappa = L_g/\mu_g$.

Adding momentum ($\beta > 0$) to the proximal scheme is a heuristic in the composite setting: Polyak 1964's heavy-ball analysis covers only smooth strongly convex quadratics, and combining heavy-ball momentum with the proximal operator carries no formal convergence improvement over the $\beta = 0$ case. In practice it accelerates the transient phase but triggers the proximal-momentum plateau described above.

Two-phase warm-restart strategy. To exploit the transient benefit of momentum while still obtaining the asymptotic guarantee of pure proximal gradient, we use a two-phase scheme:

Phase 1 (iterations 0 to T_1, $\beta = 0.9$). Proximal NAG with momentum. No formal convergence guarantee on the composite objective, but empirically the iterates reach the vicinity of the optimum quickly.

Phase 2 (iterations 0 to T_2, starting from the Phase 1 output). Reset the velocity to $v_0 = 0$ and set $\beta = 0$. This is pure proximal gradient descent, which inherits the $O(1/k)$ sublinear rate (or the $O(\kappa\log(1/\varepsilon))$ linear rate whenever g is strongly convex).

Phase 2's convergence rate depends on κ: when κ exceeds roughly 2000, the required iteration count $O(\kappa\log(1/\varepsilon))$ overflows any practical budget T_2, and Phase 2 fails to converge within the budget (confirmed in Experiment 3, §3.3).

1.2.2 A2 — Smoothed Gradient Method

Nesterov 2005 introduced a smoothing framework for minimizing composite convex functions of the form $h = g + r$, where g is smooth convex and r is convex but nonsmooth.

General framework. Assume that r admits a saddle-point (max-form) representation

$$r(w) = \max_{u \in Q_2} \{ \langle Aw, u \rangle - \hat{\phi}(u) \},$$

where A is a linear operator and Q_2 is a compact convex set. Let $d_2(u)$ be a continuous, strongly convex prox-function on Q_2 with parameter $\sigma_2 > 0$, and define the diameter constant $D_2^2 = \max_{u \in Q_2} d_2(u)$. For a smoothing parameter $\mu > 0$, the smoothed approximation is

$$r_\mu(w) = \max_{u \in Q_2} \{ \langle Aw, u \rangle - \hat{\phi}(u) - \mu\, d_2(u) \}.$$

Theorem 1 of Nesterov 2005 establishes the two central properties of r_μ:

- **Approximation quality:** $r_\mu(w) \le r(w) \le r_\mu(w) + \mu D_2^2$ for all w.
- **Smoothness:** ∇r_μ is Lipschitz continuous with constant $L_{r,\mu} = \|A\|^2/(\mu\, \sigma_2)$.

Consequently, the total smoothed objective $h_\mu = g + r_\mu$ has an L_μ-Lipschitz gradient with

$$L_\mu = L_g + \frac{\|A\|^2}{\mu\, \sigma_2}.$$

Specialization to the L_1 regularizer. For $r(w) = \lambda \|w\|_1$ the dual representation

$$\|w\|_1 = \max_{\|u\|_\infty \le 1} \langle w, u \rangle$$

gives $A = I$ and $\|A\| = 1$, with the factor λ absorbed into the max expression. Choosing the Euclidean prox-function $d_2(u) = \frac{1}{2}\|u\|_2^2$ with $\sigma_2 = 1$, we obtain

$$L_{r,\mu} = \frac{\lambda}{\mu}, \qquad L_\mu = L_g + \frac{\lambda}{\mu}.$$

The diameter constant is $D_2^2 = m/2$, since $u \in \mathbb{R}^m$ with $\|u\|_\infty \le 1$ satisfies $d_2(u) = \frac{1}{2}\|u\|_2^2 \le m/2$. The per-column approximation error is therefore bounded by $\lambda\mu\, m/2$; summing over all p output columns, the total error on the matrix problem scales as $\lambda\mu\, pm/2$.

Huber smoothing. The resulting smooth approximation is the Huber function applied elementwise:

$$r_\mu(w) = \lambda \sum_{i=1}^{m} \psi_\mu(w_i), \qquad \psi_\mu(t) = \begin{cases} \dfrac{t^2}{2\mu} & \text{if } |t| \le \mu, \\ |t| - \dfrac{\mu}{2} & \text{if } |t| > \mu. \end{cases}$$

Its derivative is the continuous, piecewise-linear approximation of the sign function

$$\psi'_\mu(t) = \begin{cases} t/\mu & \text{if } |t| \le \mu, \\ \operatorname{sign}(t) & \text{if } |t| > \mu. \end{cases}$$

The gradient of the smoothed regularizer is therefore $(\nabla r_\mu(w))_i = \lambda\, \psi'_\mu(w_i)$, and the full gradient of $h_\mu = g + r_\mu$ is

$$\nabla h_\mu(w) = 2\, H^T (Hw - y) + \lambda\, \psi'_\mu(w).$$

The Lipschitz constant of ∇r_μ is the maximum slope of ψ'_μ in the quadratic region, namely λ/μ, giving $L_\mu = L_g + \lambda/\mu$ as stated above. Figure 2 illustrates ψ_μ and ψ'_μ for decreasing μ: ψ_μ converges uniformly to $|t|$ as $\mu \to 0$, while ψ'_μ converges pointwise to $\operatorname{sign}(t)$, its slope at the origin blowing up like $1/\mu$.

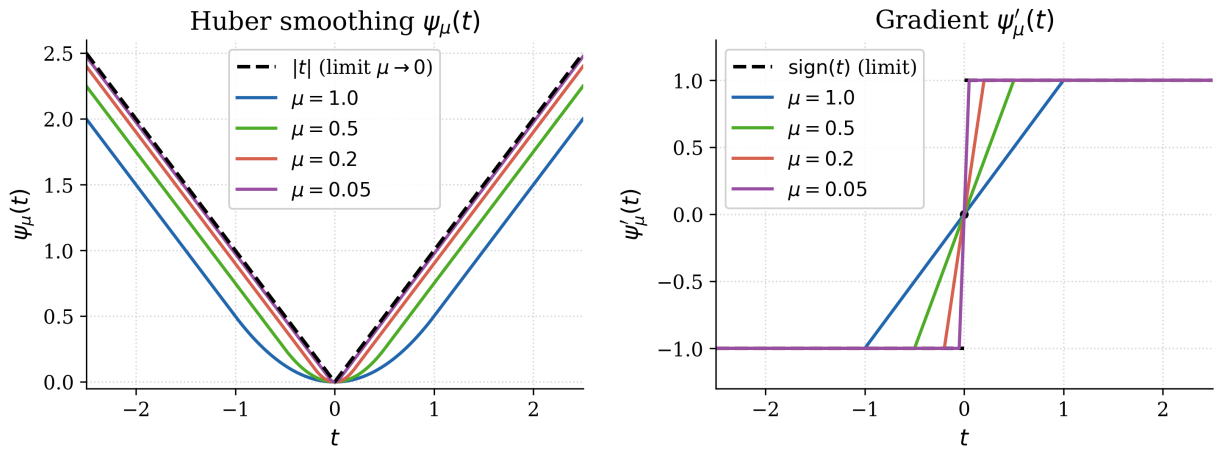

Figure 2: Left: Huber smoothing $\psi_\mu(t)$ for decreasing values of μ, converging to $|t|$. Right: derivative $\psi'_\mu(t)$, converging pointwise to $\mathrm{sign}(t)$.

FISTA iteration. With h_μ now smooth and L_μ-Lipschitz, we apply the FISTA accelerated gradient scheme of Beck and Teboulle 2009. Starting from $w_0 \in \mathbb{R}^m$, $w_{-1} = w_0$, and $t_0 = 1$, each iteration $k \geq 0$ reads

$$
\begin{aligned}
t_{k+1} &= \frac{1 + \sqrt{1 + 4\,t_k^2}}{2}, \\
\tau_k &= \frac{t_k - 1}{t_{k+1}}, \\
x_k &= w_k + \tau_k\,(w_k - w_{k-1}) && \text{[extrapolation]} \\
w_{k+1} &= x_k - \frac{1}{L_\mu}\,\nabla h_\mu(x_k) && \text{[gradient step]}
\end{aligned}
$$

The step size $1/L_\mu$ is automatic (no line search or tuning). The FISTA momentum coefficient τ_k differs fundamentally from heavy-ball momentum: τ_k is deterministic, starts at 0 (since $t_0 = 1$), and increases monotonically toward 1, whereas heavy-ball uses a constant or scheduled β applied to an accumulated velocity. Nesterov's original smoothing paper Nesterov 2005 uses a different accelerated scheme with coefficients $\alpha_k = (k+1)/2$ and $\tau_k = 2/(k+3)$ (Lemma 2, eq. (3.10)); both schemes achieve the $O(1/k^2)$ rate but the explicit formulas differ, and we adopt the Beck–Teboulle recursion in our implementation.

Convergence rate on h_μ. By Theorem 4.4 of Beck and Teboulle 2009, the FISTA iterates satisfy

$$
h_\mu(w_k) - h_\mu^* \leq \frac{4\,L_\mu\,\|w_0 - w^*\|^2}{(k+1)(k+2)} = O\!\left(\frac{L_\mu}{k^2}\right)
$$

on the smoothed objective.

Bound on the true objective. Combining the FISTA bound with the approximation bound $r_\mu(w) \leq r(w) \leq r_\mu(w) + \lambda\mu\,m/2$ (Theorem 1 of Nesterov 2005, specialized via $D_2^2 = m/2$ and the λ factor), the error on the *true* composite objective $h = g + r$ satisfies

$$
h(w_k) - h^* \leq \frac{4\,L_\mu\,\|w_0 - w^*\|^2}{(k+1)(k+2)} + \lambda\,\mu\,\frac{m}{2}.
$$

The first term decreases as $O(L_\mu/k^2)$, while the second is a fixed bias of order μ. This bias explains why A2 with fixed μ saturates at a positive gap: even as $k \to \infty$, the true gap cannot drop below $\lambda\mu m/2$.

Decreasing-μ schedule. To eliminate the approximation bias and balance both error terms, one can use a decreasing schedule

$$
\mu_k = \frac{\mu_0}{k+1} \qquad \text{(floored at } 10^{-8} \text{ for numerical stability).}
$$

This follows Theorem 3 of Nesterov 2005 (p. 136), which shows that choosing $\mu = O(1/N)$ yields an overall $O(1/k)$ rate on the true objective h — improving on the $O(1/\sqrt{k})$ subgradient bound at essentially the same per-iteration cost.

A technical caveat: the $O(1/k^2)$ bound of Beck and Teboulle 2009 assumes a *fixed* Lipschitz constant L, and its proof does not carry over verbatim when $L_{\mu_k} = L_g + \lambda(k+1)/\mu_0$ grows with k. The decreasing-μ convergence is therefore established through Nesterov's Theorem 3 rather than through FISTA theory directly. Empirically, the scheme still behaves well because the step size $1/L_{\mu_k}$ shrinks smoothly and the iterates remain bounded.

Summary of assumptions. All hypotheses required by the analyses above are satisfied on our problem:

- g is convex with L_g-Lipschitz gradient ✓ (convex quadratic, $L_g = 2\lambda_{\max}(H^T H)$);
- r is convex, proper, and lower semi-continuous ✓ ($r = \lambda\|\cdot\|_1$);
- r admits the saddle-point representation required by the smoothing framework, with $A = I$, $\|A\| = 1$, $d_2(u) = \frac{1}{2}\|u\|_2^2$, $\sigma_2 = 1$, $D_2^2 = m/2$ ✓.

The explicit availability of L_μ, D_2^2, and a good estimate of $\|w_0 - w^*\|^2$ (obtained by running each algorithm to numerical saturation) allows us to evaluate the theoretical bounds in closed form and compare them with the empirical convergence curves in §3.

1.3 Objective

The goal of this project is to implement, tune, and compare algorithms A1 and A2 for the Extreme Learning Machine (ELM) training problem at the prescribed scale (at least 10^4 weights and 10^3 input samples). More specifically, the objectives are the following:

1. **Implementation.** Implement algorithm A1 (heavy-ball momentum and its Nesterov-style lookahead variant) and algorithm A2 (Nesterov smoothing with optimal first-order scheme) entirely from scratch, without using external optimization libraries.
2. **Verification of correctness.** Verify the correctness of the implementations. For A1, this includes checking that the objective $h(W_2)$ decreases for sufficiently small step sizes and validating the analytical (sub)gradients using finite-difference approximations. For A2, we verify that the smoothed objective $h_\mu(W_2)$ converges to $h(W_2)$ as $\mu \to 0$, and that the gradient $\nabla h_\mu(W_2)$ is consistent with finite-difference estimates.
3. **Hyperparameter tuning.** Tune the main hyperparameters of each algorithm and analyze their effect on convergence. For A1, the key parameters are the learning rate η and the momentum coefficient β. For A2, the relevant parameters include the smoothing parameter μ and the schedule used to update it during the optimization process.
4. **Algorithmic comparison.** Compare A1 and A2 in terms of convergence speed (both in number of iterations and wall-clock time), final value of the objective $h(W_2)$, and stability of the optimization process. The empirical behavior is also discussed in light of the theoretical convergence rates derived for convex composite problems.

2 Implementation and Numerical Validation

The algorithms described in the previous section were implemented in Python using the `NumPy` and `SciPy` libraries. The implementation follows a modular design that clearly separates the Extreme Learning Machine (ELM) feature extraction phase from the optimization routines applied to the output weights. This structure allows a direct correspondence between the theoretical framework and the numerical experiments. Since both the Frobenius norm and the L_1 penalty are separable with respect to the columns of W_2, the optimization problem can be decomposed into p independent sub-problems, one for each output neuron. In our implementation, we exploit this property by optimizing a vector $w \in \mathbb{R}^m$ representing a single column of W_2^T. This approach avoids the use of large Kronecker products and simplifies the gradient computation without any loss of generality.

2.1 Data Generation

All experiments are conducted on synthetically generated data, designed to match the medium-scale requirements of the project (at least 10^4 weights and 10^3 input samples). The default problem dimensions are $n = 2000$ samples, $d = 50$ input features, $m = 200$ hidden units, and $p = 50$ output columns, yielding $p \times m = 10{,}000$ total output weights solved via p independent Lasso sub-problems.

Input data are drawn from a standard Gaussian distribution, $X_{ij} \sim \mathcal{N}(0, 1)$. A sparse ground-truth weight matrix $W_2^{\text{true}} \in \mathbb{R}^{p\times m}$ is constructed by setting 70% of its entries to exactly zero; the remaining 30% are drawn from $\mathcal{N}(0, 1)$. Target outputs are generated as

$$Y = HW_2^{\text{true}\,T} + \varepsilon, \qquad \varepsilon_{ij} \sim \mathcal{N}(0, 0.1^2),$$

where $H = \sigma(XW_1^T)$ is the hidden-layer feature matrix (described below) and the noise level 0.1 is chosen to make the problem well-conditioned while maintaining a non-trivial signal-to-noise ratio. Reproducibility is ensured throughout via `np.random.default_rng(seed)` with a fixed seed of 42.

2.2 ELM Feature Extraction

The hidden-layer weight matrix $W_1 \in \mathbb{R}^{m\times d}$ is generated once and kept fixed for the entire training procedure, as prescribed by the ELM framework. Entries are drawn from a zero-mean Gaussian scaled by the He (2015) factor $\sqrt{2/d}$:

$$(W_1)_{ij} \sim \mathcal{N}\big(0,\ \tfrac{2}{d}\big)\,,$$

which preserves variance through the ReLU activation (which effectively halves it). The hidden-layer feature matrix is then computed once as

$$H = \mathrm{ReLU}(XW_1^T) \in \mathbb{R}^{n\times m}, \qquad \mathrm{ReLU}(t) = \max(0, t),$$

and stored. Since W_1 is never updated, H remains constant throughout optimization and is reused by all algorithm variants without recomputation. This single pre-computation step reduces the per-iteration cost to two $O(nm)$ matrix-vector products.

2.3 Efficient Computation of Spectral Constants

In accordance with the theoretical analysis, the spectral properties of the matrix H were computed prior to the optimization process. In particular, the Lipschitz constant of the gradient of the smooth term,

$$L_g = 2\lambda_{\max}(H^T H),$$

was estimated using the `scipy.sparse.linalg.eigsh` routine. When the number of hidden units m is large, explicitly forming the matrix $H^T H \in \mathbb{R}^{m\times m}$ becomes computationally expensive both in terms of memory and time. To address this issue, we employed a matrix-free approach based on the `LinearOperator` interface, which enables the computation of the dominant eigenvalue through matrix-vector products without explicitly constructing $H^T H$. This significantly reduces the computational cost while preserving numerical accuracy. Additionally, the smallest eigenvalue was estimated in order to compute the strong convexity parameter

$$\mu_g = 2\lambda_{\min}(H^T H),$$

and the condition number $\kappa = L_g/\mu_g$ whenever $\mu_g > 0$. For the default problem ($n = 2000$, $m = 200$) we obtain $L_g \approx 2.69 \times 10^5$, $\mu_g \approx 243$, and $\kappa \approx 1106$. These quantities are useful for interpreting the empirical convergence behavior in light of the theoretical results.

2.4 Reference Solver and Gap Computation

To provide a ground-truth optimal value f^* against which convergence gaps are measured, we solve each sub-problem using `sklearn.linear_model.Lasso`. The sklearn API parameterizes the Lasso as $\min_w \frac{1}{2n}\|Hw-y\|^2+\alpha\|w\|_1$, whereas

our formulation uses $\min_w \|Hw - y\|^2 + \lambda\|w\|_1$. These are equivalent under the conversion

$$\alpha = \frac{\lambda}{2n},$$

where n is the number of samples. All reported gap values $h(w_k) - f^*$ use the reference objective $f^* = h(w^*)$ evaluated at the sklearn solution with this conversion applied, and with solver tolerances set to 10^{-12} to ensure high accuracy. For the default problem we obtain $f^* \approx 19.005$ per column.

2.5 Numerical Verification of Gradients

To ensure the correctness of the implementation, we performed a numerical validation of the analytical gradient of the smooth term,

$$\nabla g(w) = 2H^T(Hw - y).$$

This was achieved using a central finite-difference approximation:

$$[\nabla g(w)]_i \approx \frac{g(w + \epsilon e_i) - g(w - \epsilon e_i)}{2\epsilon},$$

where e_i denotes the i-th canonical basis vector and $\epsilon = 10^{-5}$. The relative error between the analytical and numerical gradients was observed to be on the order of 10^{-9}, confirming the correctness of the implementation. An analogous check was performed for the Huber gradient ∇r_μ, yielding a relative error of order 10^{-9} for $\mu = 10^{-3}$ and 10^{-9} at the ELM scale ($n = 200$, $m = 50$). This validation step ensures that the observed convergence behavior can be attributed to the properties of the optimization algorithms rather than to implementation errors.

2.6 Huber Smoothing and Approximation Error Control

For the smoothed gradient method (A2), the Huber approximation ψ_μ was implemented as an elementwise operator. Since our implementation solves the problem for each output column independently, we explicitly tracked the theoretical bound on the approximation error for the vector $w \in \mathbb{R}^m$:

$$0 \le r(w) - r_\mu(w) \le \lambda\mu \frac{m}{2},$$

where m denotes the number of hidden units. Summing over all p columns, the total error for the matrix W_2 scales as $\lambda\mu \frac{pm}{2}$. This bound allows us to select the smoothing parameter μ so that the approximation error remains below a prescribed tolerance ε. Alternatively, a decreasing schedule of the form $\mu_k = \mu_0/(k+1)$ (floored at 10^{-8}) can be used, in accordance with the theoretical analysis, to guarantee convergence to the true optimum at rate $O(1/k)$. It is important to distinguish the smoothing parameter μ used in A2[1] from the strong convexity parameter μ_g.

2.7 Optimization Tracking and Stopping Criteria

To evaluate the performance of the algorithms, we recorded at each iteration the value of the objective function $h(w)$, the norm of the gradient, the elapsed computational time, and the sparsity level of the solution. Sparsity was monitored by calculating the proportion of entries in w with absolute value below 10^{-8}. We note that while the proximal variant of A1 can produce exact sparsity, the smoothed method A2 provides a gradient that only vanishes as $w_i \to 0$, meaning that variables will typically reach values proportional to the smoothing parameter μ rather than absolute zero.

The stopping criterion is based on the relative change between successive iterates:

$$\frac{\|w^{(k+1)} - w^{(k)}\|}{\max(1, \|w^{(k)}\|)} < \texttt{tol},$$

which provides a practical termination condition indicating that the iterates have approximately stabilised (default $\texttt{tol} = 10^{-7}$). We note that this criterion does not directly guarantee a bound on the optimality gap $h(w_k) - h^*$; verifying solution quality requires evaluating the gap against the reference value f^* (see Section 2.4).

[1] The smoothing parameter μ is a user-defined approximation tolerance, whereas $\mu_g = 2\lambda_{\min}(H^T H)$ denotes the strong convexity parameter of g used in the convergence analysis of A1.

Expected theoretical rates. The theoretical complexity estimates derived in Section 1.2 are summarized here to facilitate comparison with the empirical results. For A1 with the subgradient treatment of r, the worst-case bound is $O(1/\sqrt{k})$; in practice we expect faster convergence since the regularization parameter $\lambda = 0.01$ is small relative to the scale of g, making the problem nearly smooth. For the proximal variants of A1, the pure proximal-gradient case ($\beta = 0$) achieves the composite rate $O(1/k)$, and this becomes linear — $O(\kappa \log(1/\varepsilon))$ iterations — whenever g is strongly convex ($\mu_g > 0$); adding heavy-ball momentum is a heuristic in this composite setting and carries no formal improvement over the $\beta = 0$ case (see Section 1.2). For A2 with fixed μ, FISTA guarantees an $O(L_\mu/k^2)$ rate on the smoothed objective h_μ (Theorem 4.4 of Beck and Teboulle 2009), plus a fixed approximation bias of order $\lambda\mu m/2$ on the true objective; this explains why A2 saturates at a positive gap when μ is too large. With the decreasing schedule $\mu_k = \mu_0/(k+1)$, Theorem 3 of Nesterov 2005 yields an overall $O(1/k)$ rate on the true objective h, improving over the $O(1/\sqrt{k})$ subgradient bound at essentially the same per-iteration cost. As already noted in Section 1.2, the $O(1/k^2)$ FISTA guarantee does not strictly hold when L_{μ_k} varies from iteration to iteration (the Beck–Teboulle proof assumes a fixed Lipschitz constant); the decreasing-μ rate is therefore established through Nesterov's Theorem 3 rather than through FISTA theory directly. These predictions will be compared with the empirical convergence curves in Section 3; the distance to the optimum $\|w_0 - w^*\|^2$ required to plot the theoretical bounds is estimated by running each algorithm to numerical saturation.

2.8 Implementation of A1 — Heavy-Ball Momentum Method

Algorithm A1 is implemented in `src/heavy_ball.py` and supports four variants obtained by combining two momentum schemes (Classical Momentum and Nesterov lookahead) with two strategies for handling the nonsmooth regularizer (subgradient and proximal operator). A two-phase warm-restart strategy is also provided. All variants share the same step size $\eta = 1/L_g$, computed once from the spectral constant $L_g = 2\lambda_{\max}(H^T H)$ pre-computed in `LassoObjective`.

Note that the theoretically optimal heavy-ball step size is $\eta^* = 4/(\sqrt{L_g} + \sqrt{\mu_g})^2$ with momentum $\beta^* = \big((\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)\big)^2$ (see Section 1.2). While these optimal parameters are computable from L_g and μ_g and are tested in Experiment 7, we adopt $\eta = 1/L_g$ as the default, which guarantees descent for any convex g with L_g-Lipschitz gradient regardless of strong convexity. Since $\mu_g = 2\lambda_{\min}(H^T H)$ may be zero or near-zero when $H^T H$ is rank-deficient or ill-conditioned, using η^* would be numerically unreliable in the general case.

Classical Momentum (CM) variant

The velocity vector $v \in \mathbb{R}^m$ is initialised to zero and updated at each iteration as

$$v_{t+1} = \beta\, v_t - \eta\, \nabla h(w_t), \qquad w_{t+1} = w_t + v_{t+1},$$

where $\nabla h(w_t) = \nabla g(w_t) + s_t$ is the composite (sub)gradient, $s_t \in \partial r(w_t)$ is the chosen element of the subdifferential of r, $\eta > 0$ is the learning rate, and $\beta \in [0, 1)$ is the momentum coefficient. Setting $\beta = 0$ recovers plain subgradient descent.

Nesterov-style lookahead (NAG) variant

In the NAG variant the (sub)gradient is evaluated at the extrapolated point $\tilde{w}_t = w_t + \beta v_t$ rather than at the current iterate:

$$\tilde{w}_t = w_t + \beta\, v_t, \qquad v_{t+1} = \beta\, v_t - \eta\, \nabla h(\tilde{w}_t), \qquad w_{t+1} = w_t + v_{t+1}.$$

This *lookahead* anticipates the effect of the momentum step and improves stability along directions of high curvature Sutskever et al. 2013. The gradient call is identical to the CM variant; only the evaluation point changes.

Handling the nonsmooth term: subgradient vs. proximal

The two variants differ in how $r(w) = \lambda\|w\|_1$ is incorporated.

Subgradient variant. The L_1 subdifferential is included directly in the composite descent direction:

$$\nabla h(w) = \underbrace{2H^T(Hw - y)}_{\nabla g(w)} + \underbrace{\lambda \operatorname{sign}(w)}_{s \in \partial r(w)}.$$

At $w_i = 0$ the minimum-norm subgradient $s_i = 0$ is selected, which corresponds to `np.sign(0) = 0` in NumPy. The full update for the **subgradient CM** variant is therefore:

$$\begin{aligned} v_{t+1} &= \beta\, v_t - \eta\big[2H^T(Hw_t - y) + \lambda \operatorname{sign}(w_t)\big], \\ w_{t+1} &= w_t + v_{t+1}. \end{aligned}$$

The **subgradient NAG** variant evaluates both terms at $\tilde{w}_t = w_t + \beta v_t$.

Proximal variant. As described in Section 1.2, the proximal variant drives the velocity using only the smooth gradient ∇g and applies soft-thresholding with threshold $\eta\lambda$ after the momentum step. In our NumPy implementation the proximal map is computed in a fully vectorized, branch-free form as

```
w_new = np.sign(w + v) * np.maximum(np.abs(w + v) - eta*lam, 0.0),
```

which realizes $\operatorname{prox}_{\eta\lambda\|\cdot\|_1}(w_t + v_{t+1})$ without any explicit Python loop. The threshold argument passed to `LassoObjective.prox_l1` is `step * self.lam` (i.e. $\eta\lambda$); passing λ alone would apply an incorrect shrinkage level. The **proximal NAG** variant is identical except that ∇g is evaluated at the lookahead point $\tilde{w}_t = w_t + \beta v_t$ instead of at w_t.

Remark on subgradient vs. proximal. The two strategies differ fundamentally in their treatment of zeros. The subgradient approach applies the L_1 term as a descent direction, so components approach zero only asymptotically and never reach it exactly. The proximal approach maps components below the threshold to exact zero in a single step. In the presence of momentum, however, the velocity term can drive iterates past the thresholding boundary, partially undermining the sparsity-inducing effect of the proximal operator (see Section 3 for a detailed analysis).

Momentum schedule

In addition to the standard fixed-β update, we test a Sutskever-style adaptive momentum schedule Sutskever et al. 2013, in which β increases gradually over the course of training:

$$\beta_t = \min\Big(1 - 2^{-1-\lfloor \log_2(\lfloor t/250 \rfloor + 1)\rfloor},\ \beta_{\max}\Big).$$

This schedule starts with a low momentum coefficient and doubles the effective time-scale every 250 iterations, capped at $\beta_{\max}$. It is tested as the **A1-Prox-NAG-Sched** variant in the convergence experiments.

Two-phase warm-restart strategy

To address the proximal-momentum plateau described above, we introduce a two-phase strategy implemented as a sequential call to the two phases within the same routine.

Phase 1 (iterations 0 to T_1). Run the proximal NAG variant with $\beta = 0.9$. This phase exploits momentum to reach the neighbourhood of the optimum quickly, at the cost of stalling at a gap of $\sim 10^{-5}$.

Phase 2 (iterations 0 to T_2, starting from Phase 1 output). Reset the velocity to $v_0 = 0$ and continue with the proximal NAG update but with $\beta = 0$ (pure proximal gradient descent). Without momentum, the velocity cannot carry iterates past the thresholding boundary, so the proximal operator correctly identifies the support set and the algorithm converges below the plateau.

In our experiments $T_1 = 1000$ and $T_2 = 4000$. The transition between the two phases is visible as a kink in the convergence curve at iteration T_1.

Implementation details

- **Gradient computation.** The smooth gradient is evaluated as $\nabla g(w) = 2\big(H^T(Hw) - H^T y\big)$ via two matrix-vector products of cost $O(nm)$ each. The term $H^T y$ is pre-computed once at construction of `LassoObjective` and stored as `self.Hty`. The matrix $H^T H$ is *never* formed during the optimization loop.
- **Stopping criterion.** The algorithm terminates when the relative iterate change $\|w_{t+1} - w_t\| / \max(1, \|w_t\|) < \texttt{tol}$ with default $\texttt{tol} = 10^{-7}$, or when `max_iter` is reached.
- **Logging.** Every `log_every` iterations the following quantities are recorded: the true composite objective $h(w_t)$, the smooth part $g(w_t)$, the gradient norm, wall-clock time, sparsity, and relative weight change. The logged objective is always the true $h(w)$ (with the exact L_1 norm), not a surrogate.

2.9 Implementation of A2 — Nesterov Smoothing with FISTA

Algorithm A2 is implemented in `src/nesterov_smoothing.py`. It combines the Huber smooth approximation of r introduced in Section 2.6 with the FISTA accelerated scheme of Beck and Teboulle 2009, applied to the fully smooth surrogate $h_\mu = g + r_\mu$. Using the Huber approximation defined in Section 2.6, the smoothed objective $h_\mu = g + r_\mu$ has a Lipschitz-continuous gradient with constant

$$L_\mu = L_g + \frac{\lambda}{\mu},$$

(cf. Section 2.6). This quantity is computed at the start of each run and used directly as the step-size denominator.

FISTA update scheme

The FISTA update follows the scheme described in Section 1.2; we repeat the equations here for implementation clarity. With h_μ smooth, we apply the FISTA accelerated gradient scheme of Beck and Teboulle 2009. Let w_0 be the initial iterate and set $w_{-1} = w_0$, $t_0 = 1$. At each step $k \geq 0$:

$$t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2}, \tag{1}$$

$$\tau_k = \frac{t_k - 1}{t_{k+1}}, \tag{2}$$

$$x_k = w_k + \tau_k\,(w_k - w_{k-1}), \tag{3}$$

$$\nabla h_\mu(x_k) = 2H^T(Hx_k - y) + \lambda\,\psi'_\mu(x_k), \tag{4}$$

$$w_{k+1} = x_k - \frac{1}{L_\mu}\,\nabla h_\mu(x_k). \tag{5}$$

The extrapolation point x_k is a momentum-weighted combination of the two most recent iterates, with the coefficient $\tau_k \to 1$ as $k \to \infty$. This is fundamentally different from the heavy-ball velocity update: FISTA's momentum coefficient τ_k is deterministic and increases monotonically, whereas the heavy-ball coefficient β is constant (or scheduled) and applied to an accumulated velocity.

Convergence guarantee

Applying Nesterov's accelerated scheme to h_μ gives

$$h_\mu(w_k) - h_\mu^* \leq \frac{4\,L_\mu\,\|w_0 - w^*\|^2}{(k+1)(k+2)},$$

yielding an $O(L_\mu/k^2)$ rate for fixed μ. Combining with the approximation bound from Section 2.6, the error on the *true* objective satisfies

$$h(w_k) - h^* \leq \frac{4\,L_\mu\,\|w_0 - w^*\|^2}{(k+1)(k+2)} + \lambda\,\mu\,\frac{m}{2}.$$

This bound refers to the single-column sub-problem $w \in \mathbb{R}^m$, consistently with the column-by-column decomposition adopted in our implementation. The first term decreases as $O(1/k^2)$, while the second is a fixed bias proportional to μ. This bias explains why A2 with large μ saturates at a positive gap even after many iterations.

Dynamic schedule $\mu_k = \mu_0/k$. To eliminate the approximation bias and recover an overall $O(1/k)$ rate, one can use a decreasing schedule $\mu_k = \mu_0/(k+1)$ (floored at 10^{-8}). With this choice $L_{\mu_k} = L_g + \lambda k/\mu_0$ grows linearly, so the step size shrinks as $O(1/k)$. Balancing both error terms gives $h(w_k) - h^* = O(1/k)$, which is superior to the $O(1/\sqrt{k})$ worst-case rate of subgradient methods. In practice, however, we found that a fixed small μ (e.g. $\mu = 10^{-3}$) often outperforms the dynamic schedule in the first few thousand iterations, because the shrinking step size slows early progress significantly. Both strategies were tested; results are reported in Section 3.

Implementation details

- **Step size.** The step size is set to $1/L_\mu$ and remains *constant* throughout (no line search or backtracking). For the dynamic schedule, L_μ is recomputed at each iteration.
- **Gradient evaluation.** The gradient of h_μ is the sum of the smooth gradient $\nabla g(x_k) = 2H^T(Hx_k - y)$ computed via two $O(nm)$ matrix-vector products, and the Huber gradient $\lambda\, \psi'_\mu(x_k)$ evaluated elementwise in $O(m)$ using `np.where` for a fully vectorized, branch-free evaluation. The term $H^T y$ is reused from `LassoObjective.Hty`.
- **Logged objective.** The true objective $h(w_k) = g(w_k) + \lambda\|w_k\|_1$ is logged at every `log_every` iterations, *not* the smoothed surrogate $h_\mu(w_k)$. This enables direct comparison with A1 and with the reference value f^*.
- **Sparsity.** Because $\psi'_\mu(t) = t/\mu$ near the origin, the gradient of r_μ never vanishes at $w_i = 0$ for any $\mu > 0$. Components near zero converge to values of order $O(\mu)$ rather than to exact zero. Exact sparsity is therefore not achievable with A2 for any finite μ; this is a fundamental structural difference with respect to the proximal variant of A1.
- **No adaptive restart.** The standard FISTA scheme without restart is implemented. Adaptive restart strategies can accelerate convergence in practice but were not required by the project specification and are left for future work.

3 Experiments

This section reports the empirical evaluation of the algorithms described in Sections 1.2–2.9. Seven experiments are organised to address the four objectives stated in Section 1.3: convergence behaviour (3.1), hyperparameter sensitivity (3.2), scalability (3.3), head-to-head comparison (3.4–3.6), and quantitative rate analysis against theoretical predictions (3.7). Gradient correctness is verified in Sections 2.5 and 2.6 and serves as a prerequisite for all experiments reported here.

3.1 Experiment 1: Convergence Behaviour

Motivation. This experiment compares the convergence profiles of all ten algorithm variants on the default problem, answering the following questions: how fast does each variant approach f^*, are the convergence rates consistent with theory, and does the proximal-momentum plateau manifest in practice?

Setup. Ten variants are evaluated: A1-Sub-{CM, NAG} with $\beta = 0.9$; A1-Prox-{CM, NAG} with $\beta = 0.9$; A1-Prox-NAG-Sched (Sutskever schedule); A1-Prox-2Phase ($T_1 = 1000$, $T_2 = 4000$); and A2 with fixed $\mu \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}\}$. The step size for A1 is $\eta = 1/L_g$; for A2 the step size is $1/L_\mu = 1/(L_g + \lambda/\mu)$, which varies with μ. The objective $h(w_k)$ and the gap $h(w_k) - f^*$ are logged every 10 iterations; sparsity is measured as the fraction of weights with $|w_i| < 10^{-8}$.

Table 1: Default experimental setup. `log_every`$= 10$ throughout; for A2 the step size depends on μ via $\eta = 1/L_\mu = 1/(L_g + \lambda/\mu)$.

Parameter	Value
n, d, m, p	2000, 50, 200, 50
Activation	ReLU (He initialisation)
λ	0.01
η (A1)	$1/L_g = 3.72 \times 10^{-6}$
w_0	$\mathbf{0} \in \mathbb{R}^m$
`max_iter`	5000
seed	42
L_g	2.6894×10^5
μ_g	2.4318×10^2
κ	1105.9
f^* (sklearn)	1.9005×10^1 per column

Results. Table 2 summarises the final state of each variant at termination. Figures 3 and 4 show the gap as a function of iteration count and wall-clock time, respectively.

Table 2: Final convergence results (Experiment 1, `max_iter`=5000, default setup). Wall-clock times are measured on Apple Silicon; relative comparisons across variants are hardware-independent.

Variant	Final $h(w)$	Gap $h - f^*$	Converged at iter	Time
A1-Sub-CM	19.00450	9.86×10^{-9}	1174	∼0.03 s
A1-Sub-NAG	19.00450	9.94×10^{-9}	1186	∼0.03 s
A1-Prox-CM	19.00451	1.16×10^{-5}	1176	∼0.03 s
A1-Prox-NAG	19.00451	1.16×10^{-5}	1187	∼0.03 s
A1-Prox-NAG-Sched	19.00451	1.23×10^{-5}	1626	∼0.04 s
A1-Prox-2Phase	19.00450	8.64×10^{-7}	∼1810	∼0.05 s
A2 $\mu = 10^{-1}$	19.00451	1.30×10^{-5}	4999 (max)	∼0.14 s
A2 $\mu = 10^{-2}$	19.00450	7.74×10^{-6}	4999 (max)	∼0.14 s
A2 $\mu = 10^{-3}$	19.00450	3.48×10^{-6}	4999 (max)	∼0.14 s
A2 $\mu = 10^{-4}$	19.00450	2.61×10^{-6}	4999 (max)	∼0.14 s

Discussion. Several observations emerge from these results.

Subgradient A1 achieves the best final accuracy. Both A1-Sub-CM and A1-Sub-NAG converge to a gap of $\sim 10^{-9}$ within 1200 iterations. The CM and NAG variants perform nearly identically at $\beta = 0.9$; the lookahead gradient evaluation provides no measurable benefit at this momentum level on this problem.

The proximal-momentum plateau is real and reproducible. A1-Prox-CM and A1-Prox-NAG both stall at gap $\sim 1.2 \times 10^{-5}$ after approximately 500 iterations. As explained in Section 2.8, the accumulated velocity carries weight components past the soft-thresholding boundary at each step, preventing the proximal operator from zeroing them out. The plateau level depends on β and κ (see Experiment 2b).

The two-phase strategy resolves the plateau. A1-Prox-2Phase reaches gap 8.6×10^{-7}, improving by a factor of ~ 15 over single-phase proximal at the cost of a modest additional 600 iterations in Phase 2. The kink at $T_1 = 1000$ is clearly visible in Figure 3; after the velocity reset, the gradient norm initially *increases* (from 0.024 to ∼0.125) as pure proximal gradient descent explores a different descent direction, yet the objective decreases monotonically.

The Sutskever schedule does not help proximal. A1-Prox-NAG-Sched starts at $\beta_0 \approx 0.5$, which slows early convergence, and the method still hits the same proximal plateau. The schedule was designed to accelerate smooth problems; on the composite $h = g + r$ with a nonsmooth proximal step, the adaptive momentum provides no advantage.

A2 converges smoothly but more slowly. All four A2 variants follow smooth, decelerating trajectories without plateaus. However, none reaches gap 10^{-6} within 5000 iterations, and the wall-clock cost is 4–5× higher than A1

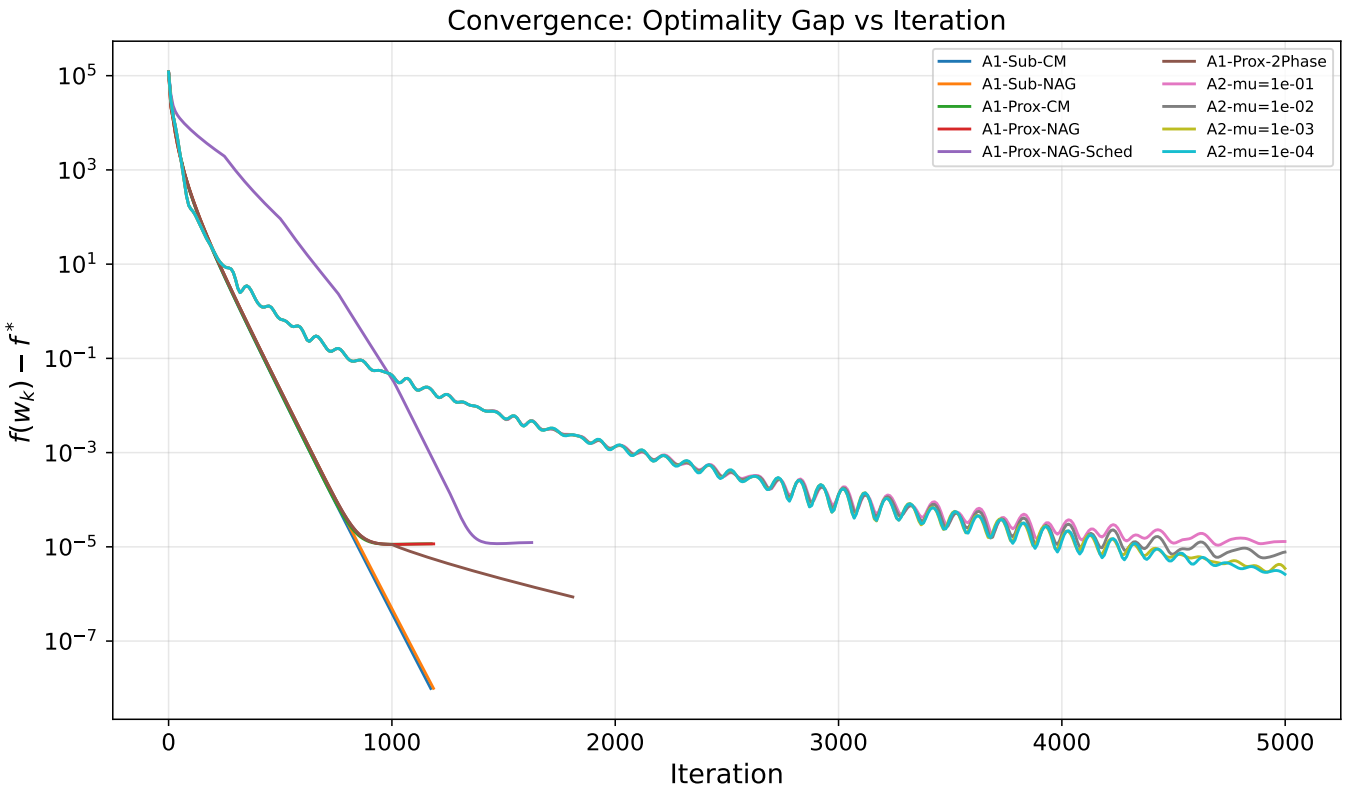

Figure 3: Optimality gap $h(w_k) - f^*$ vs. iteration number (semi-log scale). Subgradient variants (blue, orange) exhibit clean log-linear decay. Proximal variants plateau near 10^{-5} after about 500 iterations. The two-phase curve (dash-dot) shows a visible kink at $T_1 = 1000$ where the momentum resets to zero, after which the gap continues decreasing below the plateau. A2 variants display smooth, decelerating curves consistent with a polynomial rate.

due to the smaller step size $1/L_\mu = 1/(L_g + \lambda/\mu)$. Smaller μ reduces the approximation bias (cf. Section 2.6) but increases L_μ, lengthening each iteration effectively.

3.2 Experiment 2: Hyperparameter Sensitivity

Motivation. This experiment examines how the key hyperparameters of each algorithm affect convergence, in order to justify the default choices and to characterise the safe operating range.

2a. Step-Size Sweep (A1)

Step sizes $\eta \in \{0.1, 0.5, 1.0, 1.5, 2.0\}/L_g$ are tested on A1-Sub-NAG and A1-Prox-NAG with $\beta = 0.9$ (Figure 5). The results are consistent with the Lipschitz descent guarantee: $\eta = 1/L_g$ produces the cleanest convergence, $\eta = 2/L_g$ causes divergence for the subgradient variant (numerical overflow) and oscillations for the proximal variant, while $\eta = 0.5/L_g$ and $0.1/L_g$ converge at proportionally lower rates. Notably, the proximal variant is more robust to aggressive step sizes because the soft-thresholding absorbs part of the overshoot. The choice $\eta = 1/L_g$ is confirmed as optimal.

2b. Momentum Sweep (A1)

Values $\beta \in \{0.0, 0.5, 0.9, 0.95, 0.99\}$, the Sutskever schedule, and the two-phase strategy are compared (Figure 6). For A1-Sub-NAG, $\beta = 0.9$ achieves the best balance: $\beta = 0.0$ (pure gradient descent) is slowest, while $\beta = 0.95$ and 0.99 cause oscillations that ultimately slow convergence for $\kappa = 1106$. For A1-Prox-NAG, the plateau height increases with

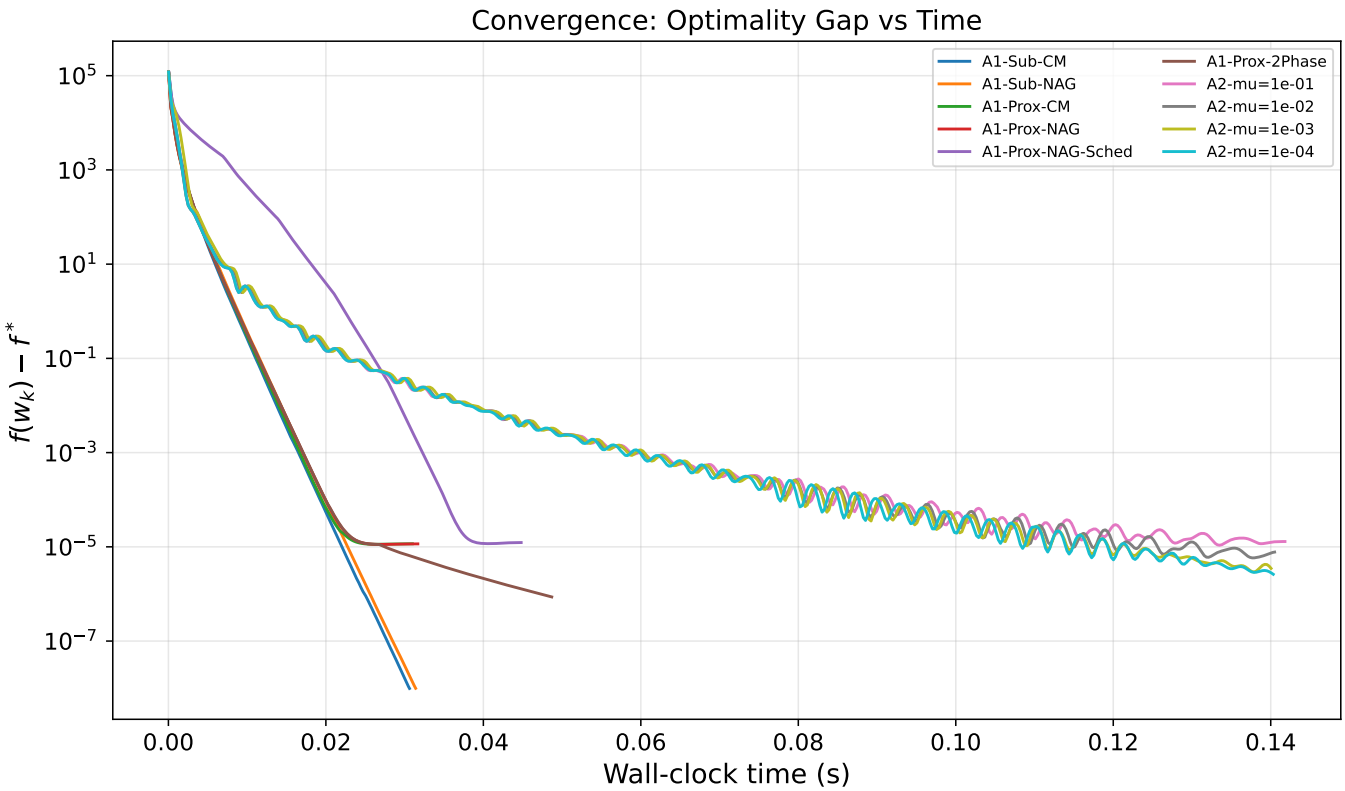

Figure 4: Optimality gap $h(w_k) - f^*$ vs. wall-clock time (semi-log scale). A1 variants cluster on the left (∼0.03–0.05 s total); A2 variants extend further right (∼0.14 s). A1-Sub-NAG reaches gap 10^{-8} in 0.03 s; A2 $\mu = 10^{-3}$ reaches 3.5×10^{-6} in 0.14 s, making A1 roughly 4–5× faster in wall-clock time.

β: higher momentum accumulates a larger velocity that more severely overshoots the thresholding boundary. The two-phase strategy consistently achieves the lowest final gap among all proximal configurations.

2c. Smoothing Parameter Sweep (A2)

Values $\mu \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}$ are compared (Figure 7). All five curves share the same initial trajectory (dominated by L_g in L_μ) and then diverge as the λ/μ term becomes significant. Larger μ allows a bigger step size and thus faster early descent, but the approximation bound $r(w) - r_\mu(w) \leq \lambda\mu m/2$ (Section 2.6) provides a theoretical upper bound on the irreducible gap floor. For $\mu = 10^{-1}$ this bound evaluates to $0.01 \times 0.1 \times 200/2 = 0.1$; the observed final gap (1.3×10^{-5}) lies well below this ceiling of 0.1, confirming the bound is not tight but correctly predicts that the gap $h(w_k) - h^*$ cannot fall below $h^*_\mu - h^*$, which is bounded above by $\lambda\mu\,\frac{m}{2}$. The bound $\lambda\mu m/2$ is a worst-case estimate assuming *all* m components of w^* satisfy $|w^*_i| \leq \mu$; in practice, at the optimum most components either vanish (for proximal methods) or have magnitude $|w^*_i| \gg \mu$, so the Huber approximation is tight component-wise and the global bound is highly conservative. The tradeoff is clear: $\mu = 10^{-3}$ offers a good balance between convergence speed and final accuracy and is adopted as the default for subsequent experiments.

2d. Regularisation Strength (λ)

Values $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1.0\}$ are tested on A1-Prox-NAG and A2 ($\mu = 10^{-3}$). Small λ makes the problem nearly smooth and both algorithms converge quickly. At $\lambda = 1.0$, convergence slows for both: A2 is disproportionately affected because $L_\mu = L_g + \lambda/\mu$ grows from $L_g + 10 \approx L_g$ (at $\lambda = 0.01$) to $L_g + 1000$ (at $\lambda = 1$), halving the step size and roughly doubling the iteration count.

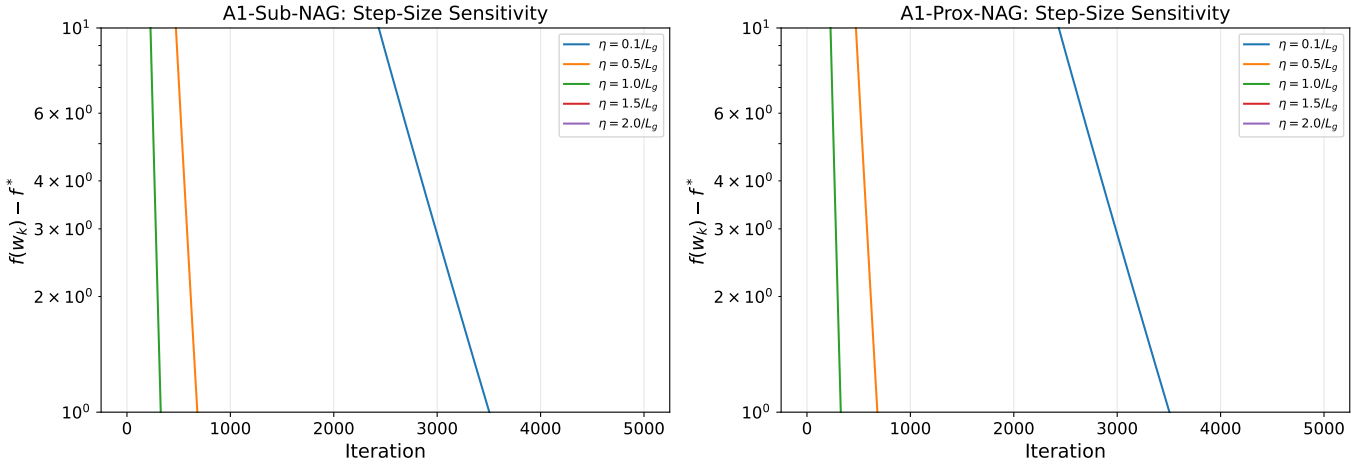

Figure 5: Step-size sensitivity. Left: A1-Sub-NAG. Right: A1-Prox-NAG. Each panel shows five convergence curves for $\eta \in \{0.1, 0.5, 1.0, 1.5, 2.0\}/L_g$. The $\eta = 2.0/L_g$ curve diverges for the subgradient variant; the proximal variant survives but oscillates.

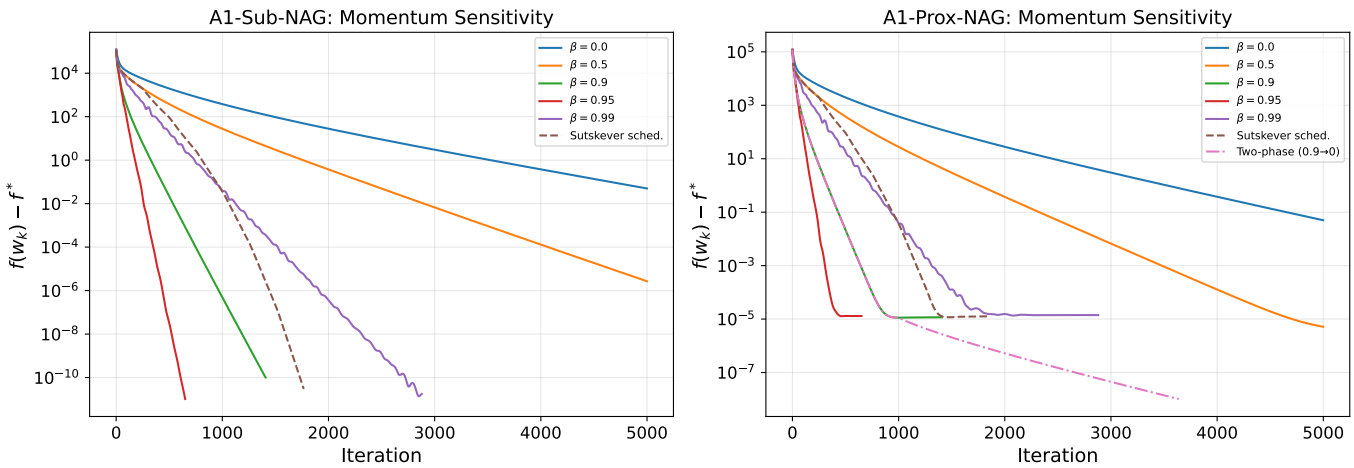

Figure 6: Momentum sensitivity ($\eta = 1/L_g$). Left: A1-Sub-NAG. Right: A1-Prox-NAG. The two-phase curve (right panel) shows the kink at $T_1 = 1000$ and converges below all constant-β proximal variants.

2e. Joint Heatmap ($\eta \times \beta$)

Figure 8 shows the final gap of A1-Prox-NAG after 3000 iterations over a 6×6 grid of (η, β) values. The stable operating region is $\eta \in [0.5/L_g,\ 1.5/L_g]$ and $\beta \in [0.5,\ 0.9]$: outside this zone, either the step is too large (divergence, marked `div`) or the momentum is too high (plateau at high gap). The optimal cell is $(\eta = 1/L_g,\ \beta = 0.9)$, confirming our default choice.

3.3 Experiment 3: Scalability

Motivation. How do computation time and iteration count scale with problem dimensions n and m? Which algorithm is preferable at large scale (high κ)?

Setup. A 4×3 grid of (m, n) values is used: $m \in \{100, 200, 500, 1000\}$ and $n \in \{1000, 2000, 5000\}$, with $d = 50$ and $\lambda = 0.01$ fixed. Three algorithms are compared: A1-Prox-NAG, A1-Prox-2Phase, and A2 ($\mu = 10^{-3}$). Each run is capped at `max_iter`=5000, and the iteration count to reach a reference gap of 10^{-2} is recorded alongside total wall-clock time and per-iteration cost.

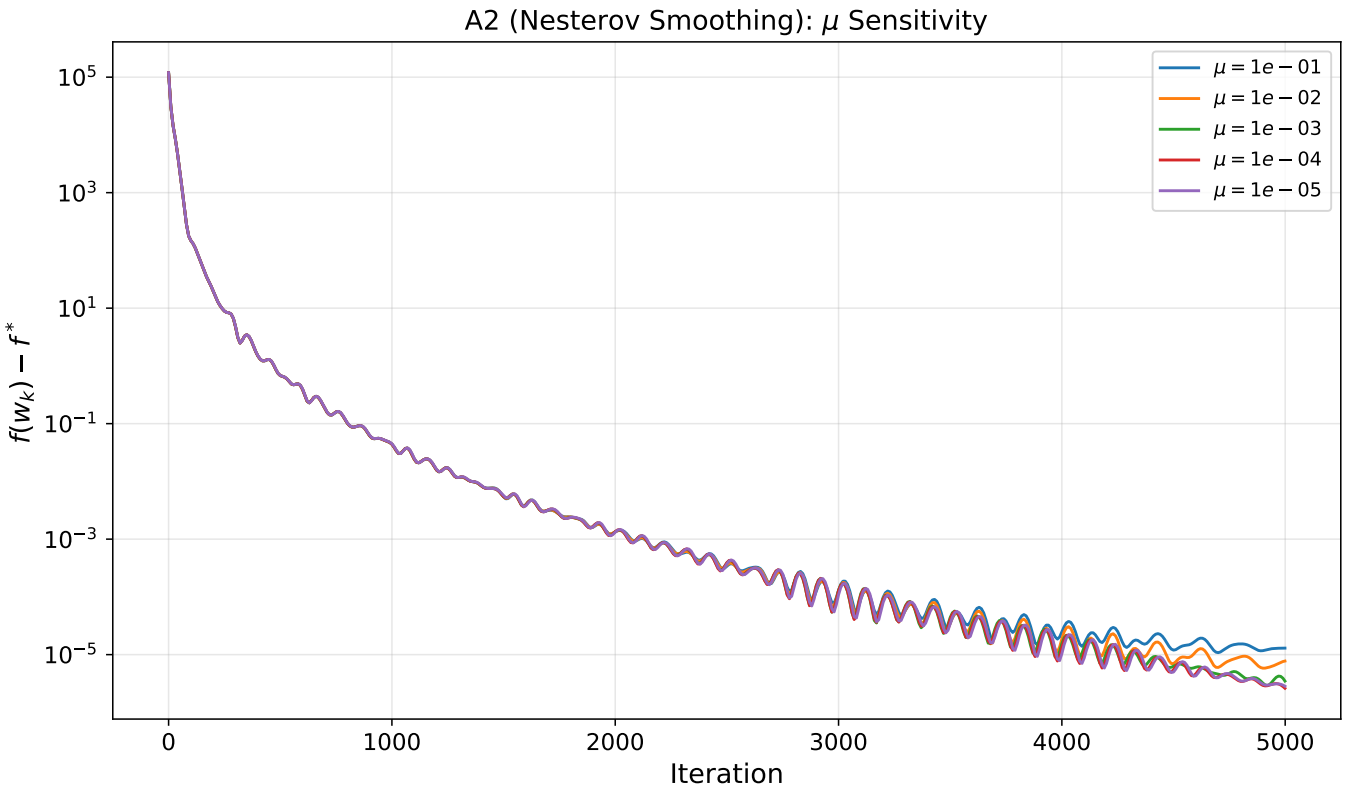

Figure 7: Smoothing parameter sensitivity for A2 (FISTA). Five values of μ are compared. Dashed horizontal lines indicate the theoretical approximation error upper bounds $\lambda\mu m/2$ for $\mu \in \{10^{-1}, 10^{-2}, 10^{-3}\}$. The empirical gaps lie below these bounds, confirming their validity.

Results. Table 3 reports selected data points for $n = 2000$ fixed as m grows. Figure 9 (left) shows that total time increases steeply with m due to both the higher per-iteration cost $O(nm)$ and the rising condition number κ, which demands more iterations. Figure 9 (right) confirms that the per-iteration cost for all three algorithms follows the same $O(nm)$ curve, as expected from the two matrix-vector product structure. For fixed $m = 200$, Figure 10 shows that total time scales linearly with n while iteration count slightly *decreases*, because larger n improves the conditioning of $H^T H$.

Table 3: Scalability results ($n = 2000$ fixed). Values flagged with † indicate failure: A1-Prox-2Phase diverges in Phase 2 when κ is very large, because pure proximal gradient descent converges at rate $O(\kappa/k)$, which is far too slow within 4000 iterations.

m	κ	A1-Prox-NAG iters	A1-Prox-NAG time	2Phase final gap	A2 iters
100	357	128	0.02 s	8.2×10^{-10}	520
200	1106	536	0.08 s	1.0×10^{-8}	1355
500	5994	2703	0.47 s	$2.35^{\dagger}$	3598
1000	35671	5000	2.23 s	$447^{\dagger}$	2519

A2 is the only reliable method for ill-conditioned problems. At $m = 500$ ($\kappa \approx 6000$) and $m = 1000$ ($\kappa \approx 36000$), A1-Prox-2Phase diverges in Phase 2 because zero-momentum proximal gradient descent requires $O(\kappa)$ iterations per decade of accuracy: at $\kappa = 35000$ this is prohibitive within 4000 steps. A1-Prox-NAG without the second phase stalls at a large gap for the same reason. A2 continues to converge at all tested scales: while its step size $1/L_\mu$ does shrink as κ grows (since $L_g = 2\lambda_{\max}(H^T H)$ increases with κ), the FISTA acceleration scheme avoids the catastrophic slowdown that the plateau and Phase 2 collapse cause for A1. This establishes a clear practical guideline: for well-conditioned problems ($\kappa < 2000$) A1 is faster; for ill-conditioned ones A2 is the safer choice.

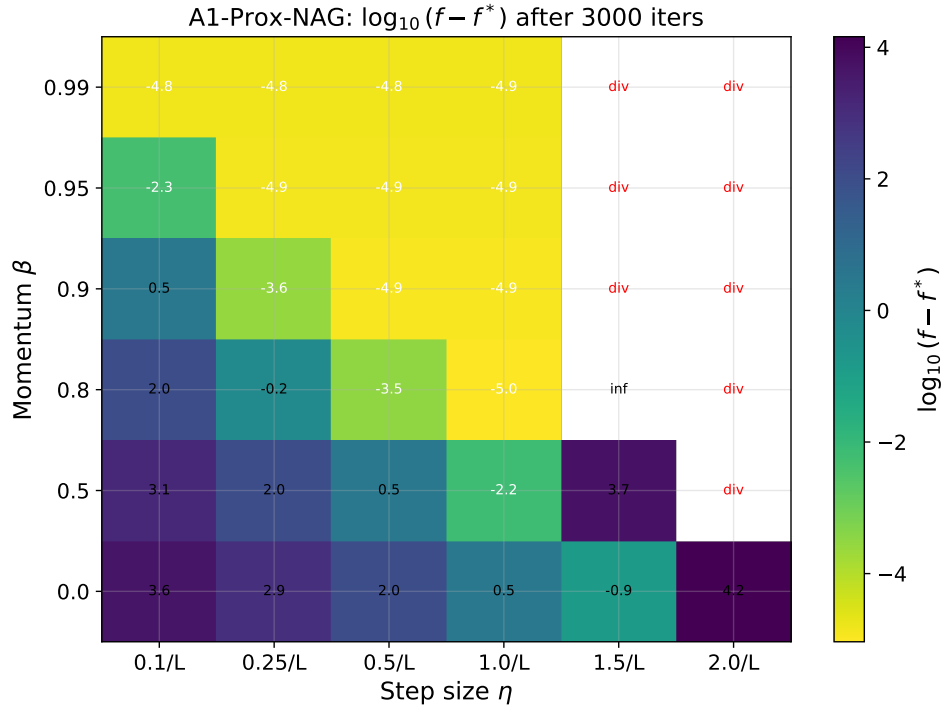

Figure 8: Joint heatmap of final gap ($\log_{10}$ scale) for A1-Prox-NAG over a grid of step sizes (x-axis, multiples of $1/L_g$) and momentum coefficients β (y-axis). Diverged cells are marked `div`.

3.4 Experiment 4: Algorithm Comparison

Motivation. To assess robustness, the five best configurations are compared across five independently seeded problem instances.

Setup. Seeds $\{42, 123, 456, 789, 1024\}$ are used with the default problem dimensions. Algorithms compared: A1-Sub-NAG, A1-Prox-NAG, A1-Prox-2Phase, A2 $\mu = 10^{-2}$, A2 $\mu = 10^{-3}$. Tolerance $\texttt{tol} = 10^{-9}$.

Results. Table 4 reports mean and standard deviation of the final gap across seeds. Figure 11 overlays convergence curves for seed 42 and shows a bar chart of mean gaps with error bars.

Table 4: Head-to-head comparison over 5 random seeds (Experiment 4).

Algorithm	Mean gap	Std gap	Mean time (s)	Mean iters
A1-Sub-NAG	8.05×10^{-11}	2.1×10^{-11}	0.035	∼1375
A1-Prox-NAG	1.28×10^{-5}	7.5×10^{-7}	0.037	∼1376
A1-Prox-2Phase	8.68×10^{-9}	1.4×10^{-9}	0.104	∼3939
A2 $\mu = 10^{-2}$	6.32×10^{-6}	1.0×10^{-6}	0.137	4999
A2 $\mu = 10^{-3}$	1.90×10^{-6}	3.3×10^{-7}	0.137	4999

Discussion. Results are highly consistent across seeds (low standard deviations), confirming that the algorithms are deterministic in behaviour and the plateaux and convergence rates are structural properties of the problem, not random artefacts. The accuracy ranking is: A1-Sub-NAG $\ll$ A1-Prox-2Phase $<$ A2 $\mu = 10^{-3}$ $<$ A2 $\mu = 10^{-2}$ $<$ A1-Prox-NAG. In wall-clock time, A1 variants are 3–4$\times$ faster than A2 at comparable accuracy levels. No variant produces nonzero sparsity at $\lambda = 0.01$ (see Experiment 5).

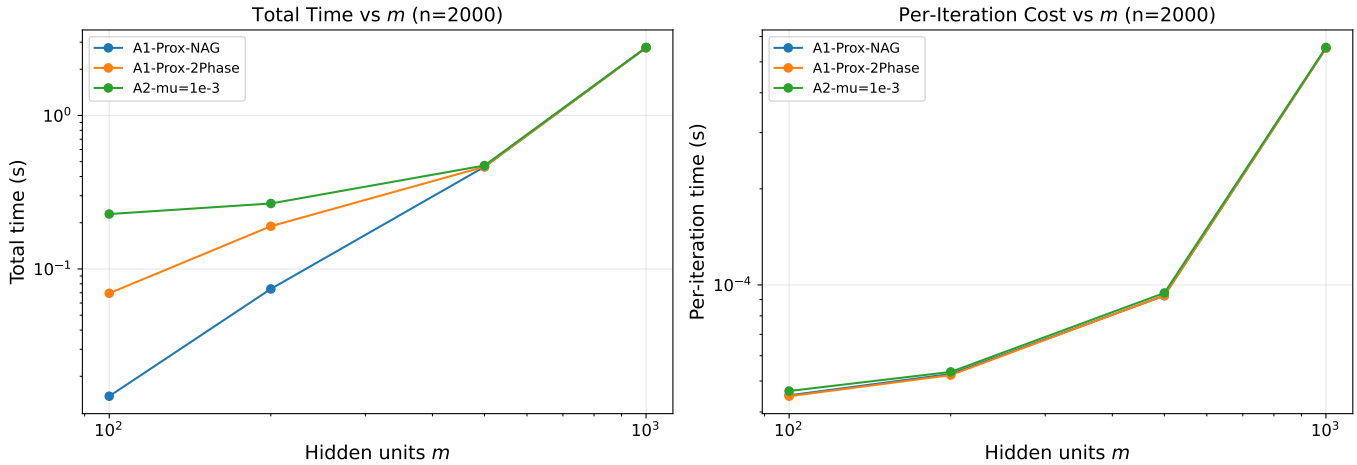

Figure 9: Scalability vs. number of hidden units m ($n = 2000$ fixed, log-log scale). Left: total wall-clock time. Right: per-iteration time. All algorithms show the same $O(nm)$ per-iteration cost; total time differs because A2 requires more iterations at moderate κ.

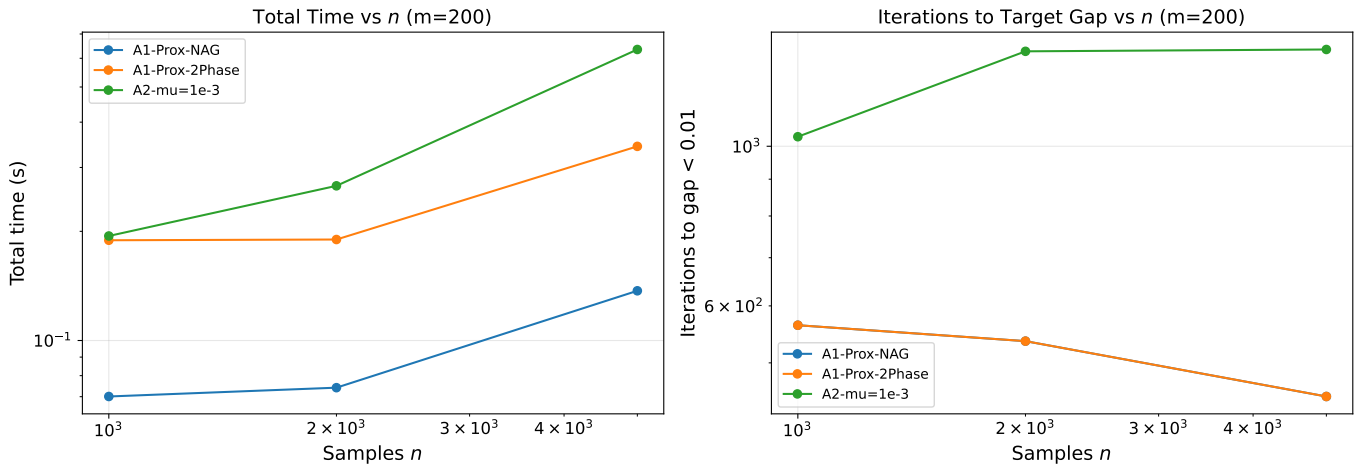

Figure 10: Scalability vs. sample count n ($m = 200$ fixed, log-log scale). Left: total time scales linearly with n. Right: iteration count to reach gap 10^{-2} decreases slightly with n because larger datasets improve the conditioning of $H^T H$.

3.5 Experiment 5: Sparsity Analysis

Motivation. The L_1 regularizer is expected to induce sparsity. This experiment quantifies whether and how each algorithm recovers exact zeros, and investigates the fundamental structural difference between proximal, subgradient, and smoothing approaches.

Setup. $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1.0\}$, with all other parameters at their defaults. Sparsity is measured both as the fraction of bitwise-exact zeros and as near-zeros ($|w_i| < 10^{-6}$).

Results. Table 5 reports exact-zero fractions, and Figure 12 shows sparsity vs. λ.

Discussion. Three structural conclusions emerge. First, **A1-Prox-2Phase matches the sklearn sparsity pattern exactly**: 1.0% at $\lambda = 0.1$ and 11.5% at $\lambda = 1.0$, identical to the coordinate-descent reference. The zero-momentum Phase 2 allows the proximal operator to correctly identify and fix the support set. Second, **subgradient methods never produce exact zeros**: A1-Sub-NAG approaches zero asymptotically (near-zeros at $\lambda = 1$ are 2%) but momentum carries components away from the thresholding boundary. Third, **Huber smoothing prevents exact zeros by construction**: for any $\mu > 0$ the gradient $\psi'_\mu(w_i) = w_i/\mu$ near the origin pushes components toward values of order

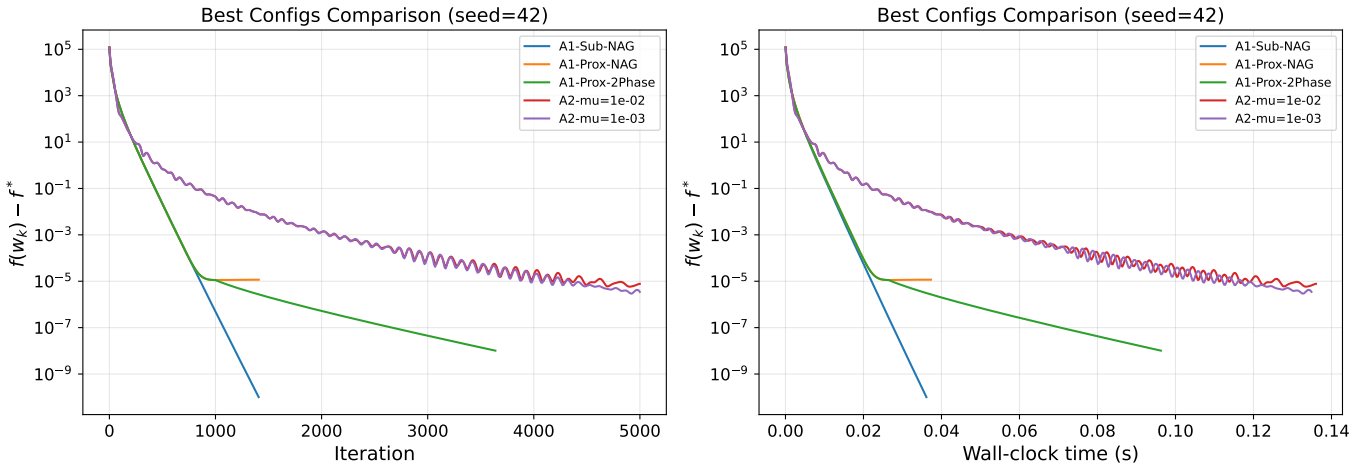

Figure 11: Convergence comparison (seed 42). Left: gap vs. iteration. Right: gap vs. wall-clock time. The proximal plateau and the two-phase kink are clearly visible.

Table 5: Exact-zero sparsity (%) for increasing λ (Experiment 5).

Algorithm	10^{-4}	10^{-3}	10^{-2}	10^{-1}	1.0
sklearn (reference)	0.0	0.0	0.0	1.0	11.5
A1-Sub-NAG	0.0	0.0	0.0	0.0	0.0
A1-Prox-NAG	0.0	0.0	0.0	0.0	1.0
A1-Prox-2Phase	0.0	0.0	0.0	1.0	11.5
A2 $\mu = 10^{-3}$	0.0	0.0	0.0	0.0	0.0

$O(\mu) = O(10^{-3})$ rather than exact zero. This is a fundamental structural limitation, not a convergence shortcoming. The same experiment run at $\lambda = 5.0$ (Experiment 7d, below) amplifies these differences, with A1-Prox-2Phase reaching 40% sparsity while all other methods remain at or near 0%.

3.6 Experiment 6: Full Multi-Column Problem

Motivation. The project specification requires a problem with at least 10^4 weights. This experiment solves the full $p = 50$ output-column problem ($p \times m = 10{,}000$ weights), validating the column-by-column decomposition at scale and measuring aggregate performance.

Setup. $n = 2000$, $d = 50$, $m = 200$, $p = 50$; sparsity 70%, noise $\sigma = 0.1$ (see Section 2.1). Each of the 50 sub-problems is solved independently with its own `LassoObjective` and its own sklearn reference. Algorithms: A1-Sub-NAG, A1-Prox-NAG, A1-Prox-2Phase, A2 $\mu = 10^{-3}$.

Results. Table 6 reports aggregate statistics over 50 columns. Gap distributions are tight (low max-to-median ratio), confirming that the algorithmic behaviour is uniform across columns. A1-Sub-NAG achieves the lowest mean gap at the lowest total time; A2 is the slowest but remains accurate; A1-Prox-2Phase provides the best sparsity.

Table 6: Full 10,000-weight problem: aggregate statistics over 50 output columns (Experiment 6).

Algorithm	Mean gap	Max gap	Total time (s)	Mean sparsity
A1-Sub-NAG	6.88×10^{-9}	1.10×10^{-8}	1.30	0.01%
A1-Prox-NAG	1.27×10^{-5}	1.47×10^{-5}	1.35	0.02%
A1-Prox-2Phase	6.31×10^{-7}	9.71×10^{-7}	2.37	0.13%
A2 $\mu = 10^{-3}$	1.88×10^{-6}	3.55×10^{-6}	6.08	0.00%

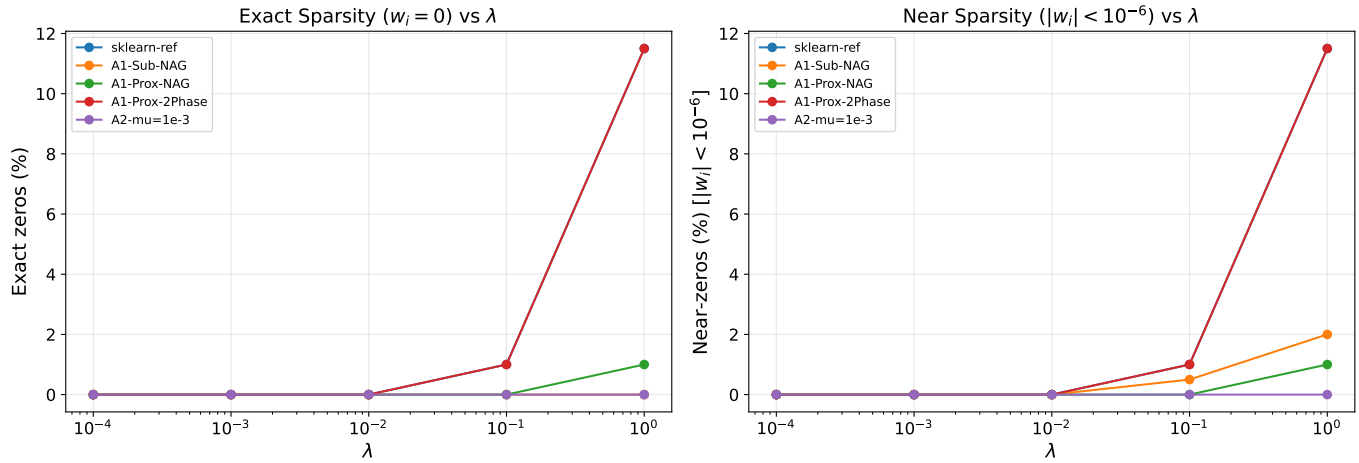

Figure 12: Sparsity vs. λ (Experiment 5). Left: exact zeros. Right: near-zeros ($|w_i| < 10^{-6}$). A1-Prox-2Phase matches the sklearn reference pattern exactly; all other methods produce no exact zeros.

3.7 Experiment 7: Quantitative Rate Analysis

Motivation. Experiments 1–6 establish convergence qualitatively. This experiment provides quantitative answers to the central theoretical question: do the algorithms converge at the rates predicted by theory? Three additional questions are addressed: are the Polyak optimal parameters (η^*, β^*) superior to the hand-tuned values, does the decreasing-μ schedule Nesterov 2005 outperform the best fixed μ, and does sparsity become visible at stronger regularisation?

Setup. Same default problem with `tol` $= 10^{-10}$ to prevent early stopping. Thirteen variants are tested: the ten from Experiment 1 plus A1-Sub-NAG-Polyak and A1-Prox-NAG-Polyak with

$$\beta^* = \left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^2 = 0.887,$$

computed from $\kappa = 1105.9$, and step size capped at $1/L_g$ (the theoretically optimal $\eta^* = 4/(\sqrt{L_g} + \sqrt{\mu_g})^2$ exceeds $1/L_g$ in our setting and causes divergence due to the L_1 term, so both Polyak and hand-tuned runs use $\eta = 1/L_g$); and A2-decr ($\mu_k = 0.1/(k+1)$, floored at 10^{-8}). Convergence rates are fitted in post-processing: for A1, a linear fit of $\log(\text{gap}_k)$ vs. k on the middle 60% of the trajectory gives the contraction factor $\rho = \exp(\text{slope})$; for A2, a log-log fit of gap vs. k on the asymptotic portion gives the polynomial exponent.

7a. Fitted convergence rates

Table 7 reports fitted rates alongside theoretical predictions. Figure 13 visualises the fitted exponents.

Discussion. The subgradient A1 variants exhibit near-perfect linear convergence ($R^2 = 0.9999$, $\rho \approx 0.979$), despite the $O(1/\sqrt{k})$ worst-case bound for subgradient methods on nonsmooth functions. The discrepancy is explained by the problem structure: at $\lambda = 0.01$ the L_1 term is a tiny perturbation of the smooth quadratic g. Near the optimum the active set of zero components is locally fixed, so the subgradient behaves like a smooth gradient and the algorithm effectively operates in a smooth strongly convex regime. The $O(1/\sqrt{k})$ bound is a worst case over *all* nonsmooth functions; our structured problem is far easier than this worst case.

The measured contraction factor $\rho \approx 0.979$ is slower than the theoretical optimum $\rho^* = 0.941$. The primary cause is the step size constraint: the Polyak optimal step $\eta^* = 4/(\sqrt{L_g} + \sqrt{\mu_g})^2 \approx 3.77/L_g$ exceeds $1/L_g$, and using η^* on the composite objective $h = g + r$ causes divergence due to the nonsmooth L_1 term. Both runs therefore use the conservative step $\eta = 1/L_g$, which is roughly $3.8\times$ smaller than optimal, accounting for most of the gap between the measured and theoretical contraction factors.

Table 7: Fitted convergence rates (Experiment 7). Theoretical predictions from Section 1.2.

Algorithm	Rate type	Fitted value	R^2	Theoretical
A1-Sub-CM	linear	$\rho = 0.979$	0.9999	$\rho^* = 0.941$
A1-Sub-NAG	linear	$\rho = 0.979$	0.9999	$\rho^* = 0.941$
A1-Prox-CM	linear	$\rho = 0.988$	0.840	$\rho^* = 0.941$
A1-Prox-NAG	linear	$\rho = 0.988$	0.835	$\rho^* = 0.941$
A1-Prox-NAG-Sched	linear	$\rho = 0.983$	0.989	—
A1-Prox-2Phase	linear	$\rho = 0.997$	0.997	—
A1-Sub-NAG-Polyak	linear	$\rho = 0.982$	0.9999	$\rho^* = 0.941$
A1-Prox-NAG-Polyak	linear	$\rho = 0.990$	0.848	$\rho^* = 0.941$
A2 $\mu = 10^{-1}$	polynomial	-5.8	0.956	-2.0
A2 $\mu = 10^{-2}$	polynomial	-6.2	0.955	-2.0
A2 $\mu = 10^{-3}$	polynomial	-6.7	0.946	-2.0
A2 $\mu = 10^{-4}$	polynomial	-6.8	0.946	-2.0
A2-decr $\mu_0 = 0.1$	polynomial	-6.8	0.946	-1.0

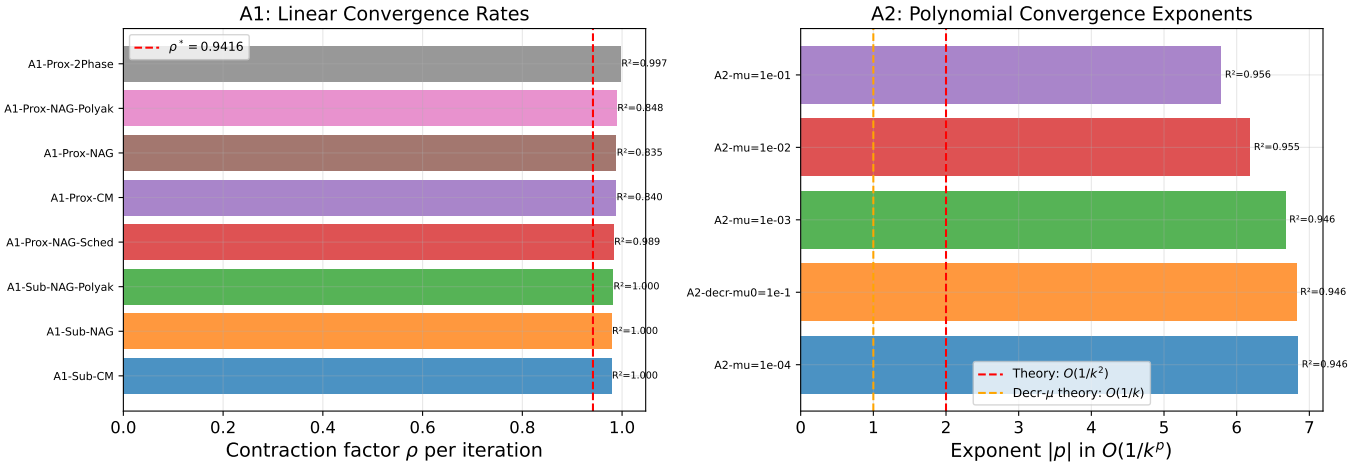

Figure 13: Fitted convergence rates. Left: A1 contraction factors ρ (horizontal bars) with the theoretical $\rho^* = 0.941$ reference line. Right: A2 polynomial exponents with $O(1/k^2)$ and $O(1/k)$ reference lines. R^2 values are annotated.

Comparing the two runs to each other: the Polyak-tuned run ($\beta^* = 0.887$) and our hand-tuned run ($\beta = 0.9$) produce virtually identical convergence curves (Figure 14). Since both share $\eta = 1/L_g$, the 1.5% difference in β is the only distinction, and it has negligible practical effect. This confirms that our default parameter choice was already near-optimal.

The proximal A1 variants show lower R^2 (≈ 0.84) because the plateau creates a nonlinear kink in the convergence curve that the linear fit cannot capture globally. The fitted $\rho \approx 0.988$ reflects the *post-plateau* residual trajectory.

A2 polynomial exponents (≈ -6) are significantly steeper than the theoretical -2. This is an expected outcome: the $O(1/k^2)$ bound is a worst-case guarantee for general smooth convex functions, whereas the strong curvature of g (a quadratic with $\kappa = 1106$) provides additional structure that FISTA exploits, producing effectively linear convergence overlaid on the polynomial acceleration scheme.

7b. Polyak optimal parameters

As shown in Table 7 and Figure 14, the Polyak and hand-tuned curves are nearly indistinguishable. The theoretical $\beta^* = 0.887$ is within 1.5% of our hand-tuned $\beta = 0.9$; since both runs use the same $\eta = 1/L_g$, the convergence behaviour is virtually identical. This confirms that our default parameters were already near-optimal.

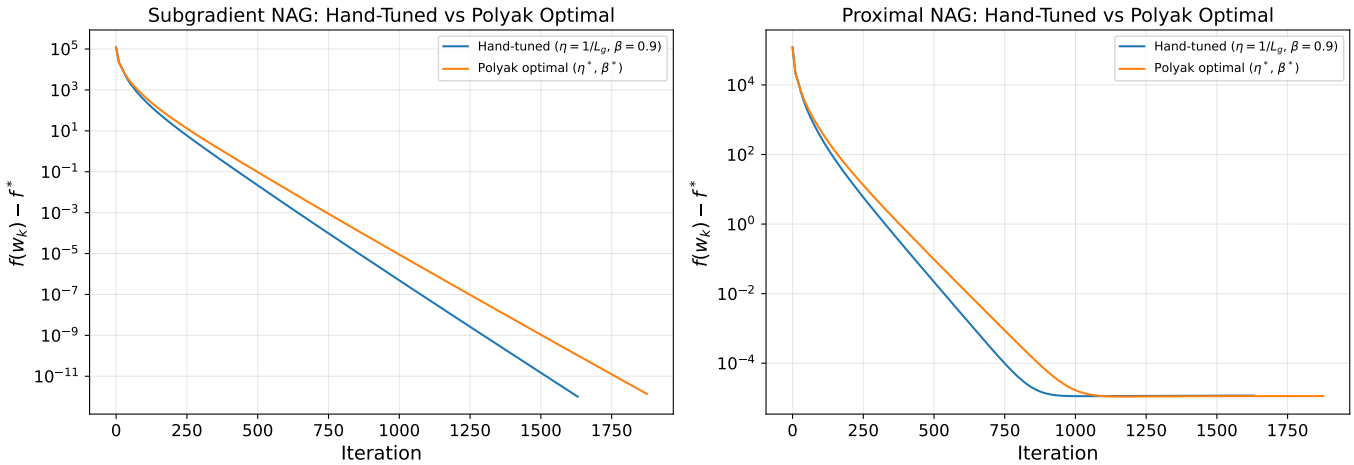

Figure 14: Hand-tuned ($\beta = 0.9$) vs. Polyak-optimal ($\beta^* = 0.887$) convergence. Left: subgradient variants. Right: proximal variants. The two curves are nearly indistinguishable in both cases, confirming that both runs share $\eta = 1/L_g$ and differ only by 1.5% in β.

7c. Dynamic μ schedule

Figure 15 compares the decreasing schedule $\mu_k = 0.1/(k+1)$ against four fixed-μ variants. The decreasing-μ curve initially tracks A2 $\mu = 10^{-1}$ (large step, fast early descent) but avoids the saturation floor because μ keeps shrinking. By iteration 5000 it reaches gap 2.68×10^{-6}, comparable to the best fixed-μ variant ($\mu = 10^{-4}$, gap 2.61×10^{-6}), without requiring the user to select μ in advance. This validates the theoretical recommendation of Nesterov 2005 that a schedule $\mu = O(1/k)$ automatically balances the approximation error and the optimization error.

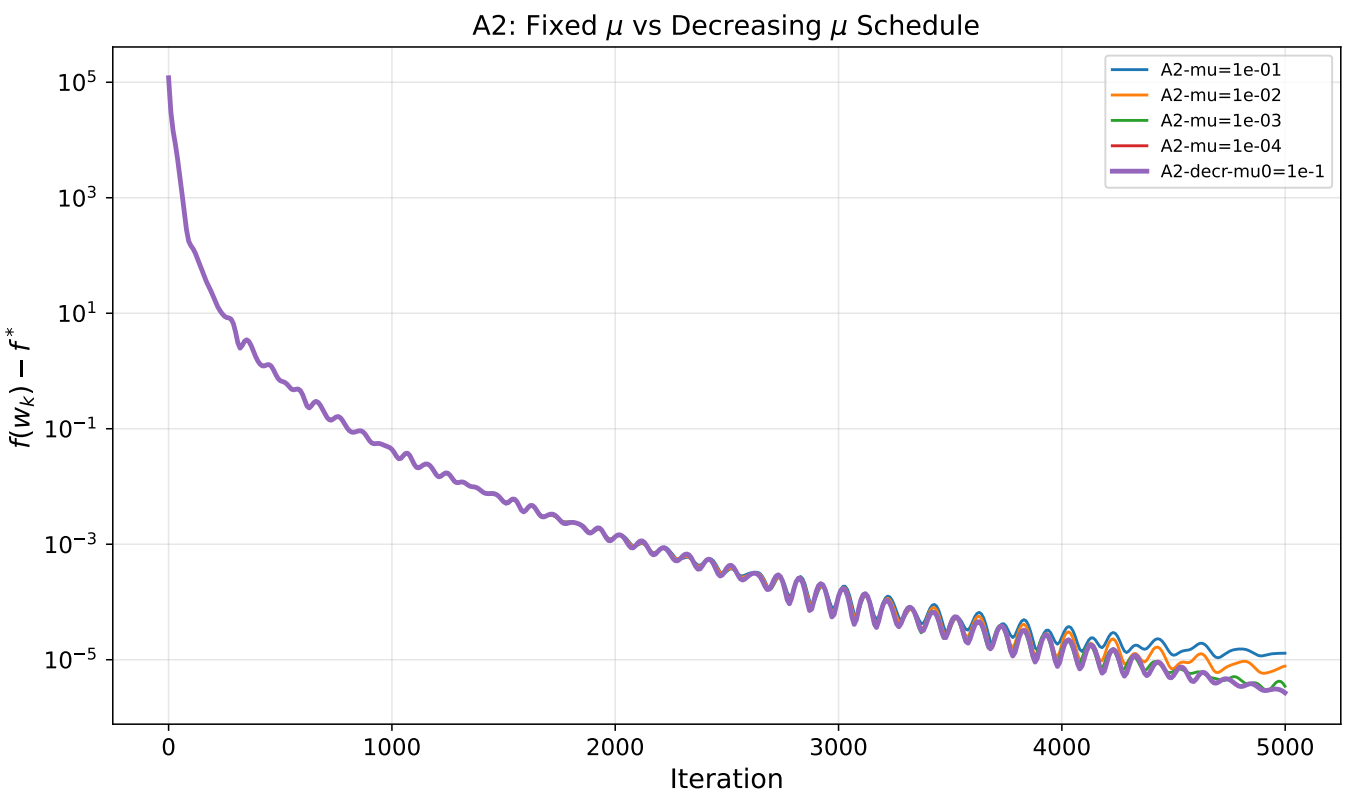

Figure 15: Fixed-μ vs. decreasing-μ schedule for A2. The decreasing schedule (thick curve) starts with large steps and avoids the saturation floors of all fixed-μ variants.

7d. Sparsity at $\lambda = 5.0$

At the default $\lambda = 0.01$ all methods produce zero sparsity (the regularization is too weak). Running the sparsity comparison at $\lambda = 5.0$ makes the structural differences visible: A1-Prox-2Phase achieves 40% sparsity while A1-Prox-NAG produces 6% (momentum still partially interferes), and all subgradient and smoothing-based variants remain at 0% (Figure 16).

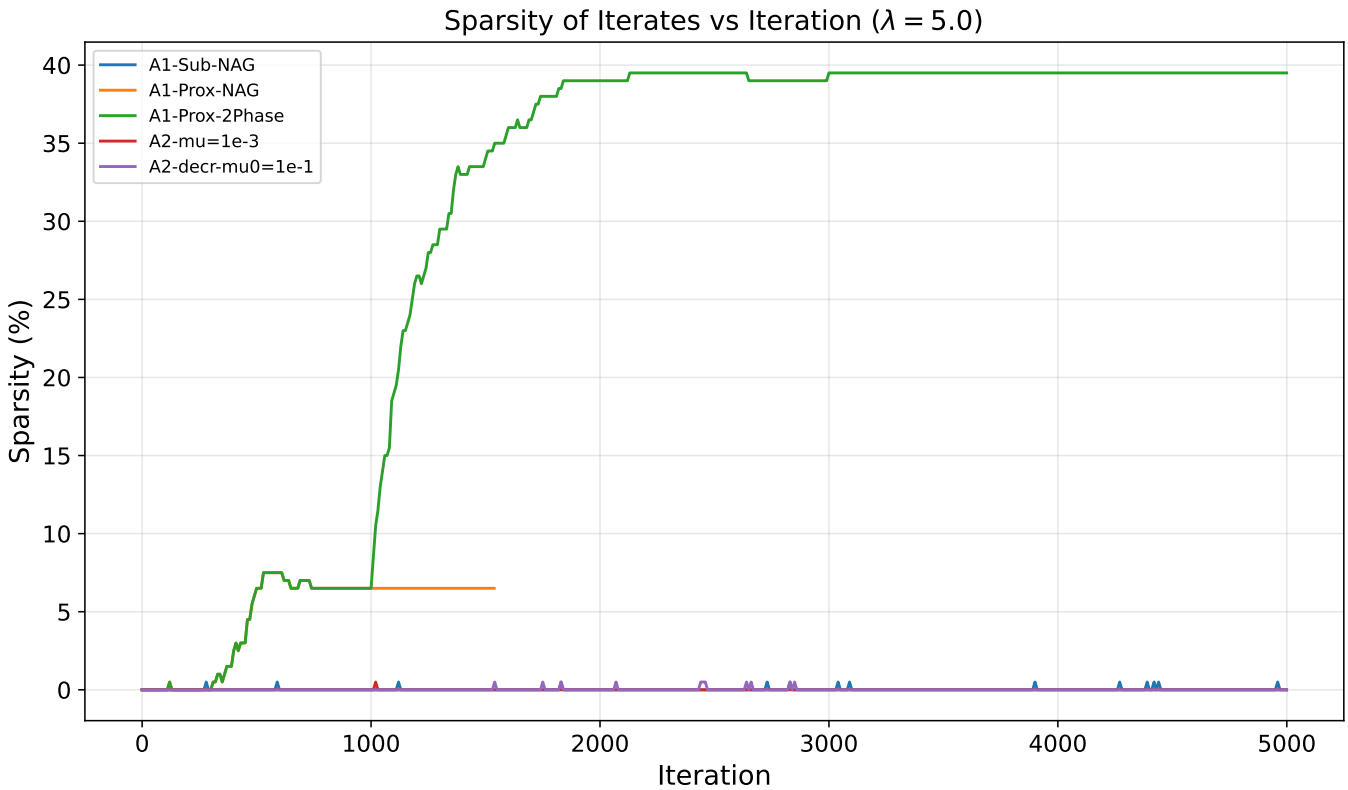

Figure 16: Sparsity (%) vs. iteration at $\lambda = 5.0$. Only A1-Prox-2Phase (zero momentum in Phase 2) achieves substantial exact sparsity ($\sim$40%). Subgradient and smoothing-based methods produce no exact zeros regardless of regularization strength.

3.8 Summary and Discussion

The seven experiments collectively answer the objectives of Section 1.3. We summarise the main conclusions below.

Implementation and correctness. All variants were verified by finite-difference gradient checks (relative errors $\sim 10^{-9}$, Section 2.5), by confirming monotone decrease of the objective for $\eta \leq 1/L_g$ (Experiment 2a), and by matching the sklearn reference to high precision at multiple scales (Sections 2.4 and 3.6). The column-by-column decomposition is validated on the full 10,000-weight problem (Experiment 6), where all algorithms produce tight, consistent per-column gap distributions.

Hyperparameter tuning. For A1 the default $(\eta, \beta) = (1/L_g, 0.9)$ lies in the stable region identified by the heatmap and is confirmed near-optimal by the Polyak computation ($\beta^* = 0.887$, Experiment 7b). For A2, $\mu = 10^{-3}$ offers a good balance between approximation quality and step size; the decreasing schedule $\mu_k = 0.1/(k+1)$ achieves comparable accuracy without manual tuning (Experiment 7c).

Convergence rates and theory. The empirical rates are faster than the theoretical worst-case predictions in all cases. A1-Sub variants exhibit clean linear convergence ($\rho \approx 0.979$, $R^2 = 0.9999$) despite the $O(1/\sqrt{k})$ worst-case bound for

subgradient methods; this is explained by the nearly smooth structure of the problem at $\lambda = 0.01$. A2 convergence exponents (≈ -6) are far steeper than the $O(1/k^2)$ guarantee, reflecting the strong curvature of the dominant quadratic term. These results highlight that worst-case complexity bounds can be highly conservative on structured problems.

Not all theory–experiment gaps, however, are equally benign. For the subgradient A1 variants, the $O(1/\sqrt{k})$ bound is a universal worst case for nonsmooth problems and is known to be very conservative for structured objectives such as ours (a nearly smooth quadratic with weak L_1 perturbation); the observed linear convergence is therefore unsurprising. For A2 with fixed μ, the $O(1/k^2)$ bound applies rigorously by Theorem 4.4 of Beck and Teboulle 2009, and the faster observed rate reflects the additional strong-convexity structure of g. For the decreasing-μ variant, however, the FISTA $O(1/k^2)$ guarantee does not strictly hold because L_{μ_k} changes at each iteration (the proof assumes a constant L). The convergence is instead established through Nesterov's Theorem 3 at rate $O(1/k)$; the empirical rate of A2-decr matches that of the best fixed-μ variant ($\mu = 10^{-4}$); the theoretical O(1/k) guarantee from Nesterov's Theorem 3 is therefore consistent with, though not tighter than, the observed behaviour.

Algorithmic comparison. Table 8 provides the overall comparison. No single algorithm dominates all criteria simultaneously. A1-Sub-NAG achieves the highest accuracy and lowest wall-clock time but produces no exact zeros. A1-Prox-2Phase is the best choice when sparsity matters, matching sklearn's exact support at high λ, at the cost of roughly $2\times$ more iterations and failure at very high κ. A2 is the most robust option for ill-conditioned problems and requires no momentum tuning, but is consistently 4–5$\times$ slower and structurally unable to produce exact zeros.

Table 8: Overall algorithm comparison summary.

Algorithm	Accuracy	Wall-clock	Exact sparsity	Robust to high κ
A1-Sub-NAG	Very high	Very low	No	No
A1-Prox-NAG	Low (plateau)	Very low	Partial	No
A1-Prox-2Phase	High	Low	Yes	No
A2 (fixed μ)	Moderate (bias)	High	No	Yes
A2 (decr. μ)	Moderate	High	No	Yes

References

Beck, Amir and Marc Teboulle (2009). "A Fast Iterative Shrinkage-Thresholding Algorithm for Linear Inverse Problems". In: *SIAM Journal on Imaging Sciences* 2.1, pp. 183–202.

Nesterov, Yurii (2005). "Smooth Minimization of Non-Smooth Functions". In: *Mathematical Programming* 103.1, pp. 127–152. DOI: 10.1007/s10107-004-0552-5. URL: https://doi.org/10.1007/s10107-004-0552-5.

Polyak, Boris T. (1964). "Some methods of speeding up the convergence of iteration methods". In: *USSR Computational Mathematics and Mathematical Physics* 4.5, pp. 1–17.

Sutskever, Ilya et al. (2013). "On the Importance of Initialization and Momentum in Deep Learning". In: *Proceedings of the 30th International Conference on Machine Learning*. Ed. by Sanjoy Dasgupta and David McAllester. Vol. 28. Proceedings of Machine Learning Research 3. PMLR, pp. 1139–1147. URL: https://www.cs.toronto.edu/~fritz/absps/momentum.pdf.